Language Modeling from Scratch

A Comprehensive Guide to Building Large Language Models

Based on Stanford CS336

CS336 Textbook Project

April 5, 2026

Preface

This book is a comprehensive companion to Stanford's CS336: Language Modeling from Scratch. It follows the course's "from scratch" philosophy—building every component of a modern language model, from tokenization to alignment, with full mathematical rigor and detailed implementation guidance.

Who This Book Is For

You are a computer science undergraduate (junior or senior level) who:

- Can program fluently in Python
- Has taken introductory courses in machine learning and linear algebra
- May have used AI tools like ChatGPT or GitHub Copilot
- Does *not* necessarily know NLP, linguistics, CUDA, or distributed systems

If some of those prerequisites feel shaky, Part 0 provides self-contained refreshers on everything you need.

How to Read This Book

The book is organized into six parts plus appendices:

Part 0: Prerequisites Probability, linear algebra, PyTorch, GPU basics, and linguistics background

Part I: Foundations The language modeling problem, tokenization, and the path from n-grams to neural models

Part II: Transformer Internals Attention, Transformer blocks, normalization, and architectural variants

Part III: Systems Resource accounting, GPUs, kernels, parallelism, and inference

Part IV: Scaling and Evaluation Scaling laws, model recipes, and benchmarking

Part V: Data Data sources, filtering, deduplication, and mixing

Part VI: Alignment Instruction tuning, RLHF, DPO, reasoning RL, and safety

You can read sequentially, or follow one of the suggested reading paths in Chapter 0.

Neuroscience Analogies

Throughout this book, we use analogies from neuroscience and brain science to build intuition. Every analogy box clearly states where the analogy helps, where it breaks down, and what intuition you should keep. We never imply that language models are brains—but the parallels can sharpen your understanding of both.

Acknowledgments

This book builds on the materials, lectures, and assignments created by Tatsunori Hashimoto, Percy Liang, and the CS336 teaching staff. We are grateful for their vision of teaching language modeling from scratch.

Contents

Preface **iii**

I Orientation and Prerequisites **1**

1 How to Use This Book **3**
1.1 The "From Scratch" Philosophy . . . 3
1.2 How to Read This Book . . . 3
1.2.1 Sequential Reading . . . 3
1.2.2 "I Know ML" Path . . . 4
1.2.3 Systems Focus . . . 4
1.2.4 Assignment Companion . . . 4
1.3 Notation Conventions . . . 4
1.4 Neuroscience Analogies . . . 4
1.5 Notation Table . . . 5

2 Probability, Entropy, and Information Theory **7**
2.1 Big Picture . . . 7
2.2 Probability Distributions over Tokens . . . 8
2.3 Entropy: Measuring Uncertainty . . . 8
2.3.1 Definition and Intuition . . . 8
2.3.2 Key Properties . . . 9
2.3.3 Worked Example: Computing Entropy . . . 9
2.4 Cross-Entropy: Measuring Model Quality . . . 10
2.4.1 Definition . . . 10
2.4.2 Cross-Entropy as a Training Loss . . . 10
2.5 KL Divergence: The Gap Between Distributions . . . 11
2.6 Maximum Likelihood and Cross-Entropy . . . 12
2.7 Perplexity: The Standard Evaluation Metric . . . 12
2.7.1 Worked Example: Perplexity Calculation . . . 13

2.8 Shannon's Game and the Entropy Rate of English . . . 14
2.9 Common Misconceptions . . . 14
2.10 Exercises . . . 15
2.11 Further Reading . . . 16

3 Linear Algebra and Matrix Calculus **17**
3.1 Big Picture . . . 17
3.2 Formal Setup and Notation . . . 18
3.3 Matrix Multiplication Views . . . 18
3.3.1 Block Matrix Multiplication . . . 18
3.4 Einstein Summation Notation (Einsum) . . . 19
3.5 Matrix Calculus Basics . . . 19
3.5.1 The Gradient of a Scalar w.r.t a Vector . . . 19
3.5.2 The Chain Rule via Jacobians . . . 19
3.5.3 Key Matrix Calculus Rules . . . 20
3.6 Worked Example: Backprop through a Linear Layer . . . 20
3.7 Systems and Implementation Consequences . . . 20
3.8 Common Misconceptions . . . 21
3.9 Exercises . . . 21
3.10 Further Reading . . . 22

4 PyTorch and Tensor Computation **23**
4.1 Big Picture . . . 23
4.2 Formal Setup and Notation . . . 24
4.3 Strides, Views, and Contiguity . . . 24
4.4 Broadcasting . . . 24
4.5 Autograd: Automatic Differentiation . . . 25
4.6 Worked Example: The Gradient Accumulation Pattern . . . 25
4.7 Systems and Implementation Consequences . . . 26
4.8 Common Misconceptions . . . 27
4.9 Exercises . . . 27
4.10 Further Reading . . . 27

5 CUDA and GPU Execution **29**
5.1 Big Picture . . . 29
5.2 Formal Setup and Notation . . . 30
5.3 The Compute Hierarchy: Grids, Blocks, Threads . . . 30
5.4 The Memory Hierarchy and the Bandwidth Wall . . . 30
5.4.1 The Bandwidth Bottleneck . . . 31

5.5 Arithmetic Intensity . . . 31
5.6 Worked Example: Kernel Overhead . . . 31
5.7 Systems and Implementation Consequences . . . 32
5.8 Common Misconceptions . . . 32
5.9 Exercises . . . 33
5.10 Further Reading . . . 33

6 Minimal Linguistics Background **35**
6.1 Big Picture . . . 35
6.2 Formal Setup and Notation . . . 36
6.3 Morphology and Typology . . . 36
6.4 Syntax vs. Semantics . . . 36
6.5 Zipf's Law . . . 37
6.6 Worked Example: Checking Zipf on a Small Sample . . . 37
6.7 Systems and Implementation Consequences . . . 38
6.8 Common Misconceptions . . . 38
6.9 Exercises . . . 39
6.10 Further Reading . . . 39

II Language Modeling Foundations **41**

7 The Language Modeling Problem **43**
7.1 Big Picture . . . 43
7.2 Formal Setup and Notation . . . 44
7.3 Autoregressive Decomposition . . . 44
7.4 Scoring versus Generation . . . 44
7.4.1 Scoring . . . 45
7.4.2 Generation . . . 45
7.5 Worked Example: Computing Sequence Probability . . . 45
7.6 Systems and Implementation Consequences . . . 46
7.7 Common Misconceptions . . . 46
7.8 Exercises . . . 47
7.9 Further Reading . . . 47

8 Tokenization **49**
8.1 Big Picture . . . 49
8.2 Formal Setup and Notation . . . 50
8.3 Byte-Pair Encoding (BPE) . . . 50
8.3.1 The BPE Training Algorithm . . . 50

8.3.2 Encoding with BPE . . . 51
8.4 The Unigram Language Model Tokenizer . . . 51
8.5 Worked Example: BPE Training . . . 51
8.6 Systems and Implementation Consequences . . . 52
8.7 Common Misconceptions . . . 53
8.8 Exercises . . . 54
8.9 Further Reading . . . 54

9 From N-Grams to Neural Language Models 55
9.1 Big Picture . . . 55
9.2 Formal Setup and Notation . . . 56
9.3 The N-Gram Language Model . . . 56
9.3.1 The Curse of Dimensionality . . . 56
9.4 Worked Example: Trigram Calculation . . . 56
9.5 The Neural Probabilistic Language Model (NPLM) . . . 57
9.6 The Recurrent Era and Its Failure Modes . . . 57
9.7 Systems and Implementation Consequences . . . 58
9.8 Common Misconceptions . . . 59
9.9 Exercises . . . 59
9.10 Further Reading . . . 60

III Transformer Internals 61

10 Attention Mechanisms 63
10.1 Big Picture . . . 63
10.2 Formal Setup and Notation . . . 64
10.3 Scaled Dot-Product Attention . . . 64
10.3.1 The Pre-Softmax Scaling Factor . . . 64
10.4 Causal Masking . . . 64
10.5 Multi-Head Attention (MHA) . . . 65
10.6 Worked Example: Attention Matrix Shapes . . . 65
10.7 Systems and Implementation Consequences . . . 66
10.8 Common Misconceptions . . . 66
10.9 Exercises . . . 67
10.10 Further Reading . . . 67

11 Transformer Blocks from Scratch 69
11.1 Big Picture . . . 69
11.2 Formal Setup and Notation . . . 70

11.3 The Residual Connection . . . 70
11.4 Layer Normalization: Pre-Norm vs Post-Norm . . . 70
11.5 The Position-Wise Feed-Forward Network (FFN) . . . 71
11.6 Assembling the Final Block . . . 71
11.7 Worked Example: Parameter Counting . . . 72
11.8 Systems and Implementation Consequences . . . 72
11.9 Common Misconceptions . . . 73
11.10 Exercises . . . 73
11.11 Further Reading . . . 74

12 Position Embeddings: RoPE and ALiBi **75**
12.1 Big Picture . . . 75
12.2 Formal Setup and Notation . . . 76
12.3 Absolute Position Embeddings . . . 76
12.3.1 Sinusoidal Vectors . . . 76
12.3.2 Learned Absolute Vectors . . . 76
12.4 Rotary Position Embeddings (RoPE) . . . 76
12.5 ALiBi: Attention with Linear Biases . . . 77
12.6 Worked Example: Implementing RoPE efficiently . . . 77
12.7 Systems and Implementation Consequences . . . 78
12.8 Common Misconceptions . . . 78
12.9 Exercises . . . 79
12.10 Further Reading . . . 79

13 Initialization and Pre-training Setup **81**
13.1 Big Picture . . . 81
13.2 Formal Setup and Notation . . . 82
13.3 Variance Preservation . . . 82
13.4 Transformer-Specific Initialization . . . 82
13.4.1 Standard Deviation . . . 82
13.4.2 Residual Deep Scaling . . . 83
13.5 The Training Loop Setup . . . 83
13.6 Worked Example: Checking Initialization in PyTorch . . . 83
13.7 Systems and Implementation Consequences . . . 84
13.8 Common Misconceptions . . . 84
13.9 Exercises . . . 85
13.10 Further Reading . . . 85

14 Mixture of Experts **87**
14.1 Big Picture . . . 87
14.2 Formal Setup and Notation . . . 88
14.3 The Router: From Dense to Sparse . . . 89
14.3.1 Gating Function . . . 89
14.3.2 Top-k Selection . . . 89
14.3.3 Top-1 vs. Top-2 Routing . . . 89
14.4 Capacity Factor and Token Dropping . . . 90
14.4.1 The Load Imbalance Problem . . . 90
14.4.2 Capacity Factor . . . 90
14.4.3 Token Dropping . . . 90
14.5 Auxiliary Load Balancing Loss . . . 91
14.5.1 The Switch Transformer Formulation . . . 91
14.5.2 Aux-Loss-Free Approaches . . . 91
14.6 Expert Collapse and Training Instability . . . 91
14.7 Dispatch and Combine: Implementation . . . 92
14.8 All-to-All Communication . . . 92
14.9 Expert Parallelism . . . 93
14.10 Worked Example: Routing 8 Tokens Through 4 Experts . . . 93
14.11 Systems and Implementation Consequences . . . 94
14.11.1 Training: Memory and Throughput . . . 94
14.11.2 Inference: Latency, Throughput, and Cache . . . 95
14.11.3 Sparse vs. Dense Tradeoffs . . . 95
14.12 The MoE Landscape . . . 96
14.12.1 Sparsely-Gated MoE (Shazeer et al., 2017) . . . 96
14.12.2 GShard (Lepikhin et al., 2020) . . . 96
14.12.3 Switch Transformer (Fedus et al., 2022) . . . 96
14.12.4 Mixtral (Mistral, 2024) . . . 96
14.12.5 DeepSeek-V2 and V3 . . . 97
14.13 Fine-Tuning Risks . . . 97
14.14 MoE, Long Context, and Deployment Cost . . . 97
14.15 Common Misconceptions . . . 98
14.16 Exercises . . . 99
14.17 Further Reading . . . 100

IV Systems and Implementation **101**

15 Resource Accounting **103**

15.1 Big Picture . . . 103
15.2 Formal Setup and Notation . . . 103
15.3 Main Ideas . . . 103
15.4 Worked Example . . . 103
15.5 Systems and Implementation Consequences . . . 103
15.6 Common Misconceptions . . . 104
15.7 Exercises . . . 104
15.8 Further Reading . . . 104

16 GPUs, TPUs, and Memory Hierarchy 105
16.1 Big Picture . . . 105
16.2 Formal Setup and Notation . . . 105
16.3 Main Ideas . . . 105
16.4 Worked Example . . . 105
16.5 Systems and Implementation Consequences . . . 105
16.6 Common Misconceptions . . . 106
16.7 Exercises . . . 106
16.8 Further Reading . . . 106

17 Kernels, CUDA, Triton, and FlashAttention 107
17.1 Big Picture . . . 107
17.2 Formal Setup and Notation . . . 107
17.3 Main Ideas . . . 107
17.4 Worked Example . . . 107
17.5 Systems and Implementation Consequences . . . 108
17.6 Common Misconceptions . . . 108
17.7 Exercises . . . 108
17.8 Further Reading . . . 108

18 Parallelism and Distributed Training 109
18.1 Big Picture . . . 109
18.2 Formal Setup and Notation . . . 109
18.3 Main Ideas . . . 109
18.4 Worked Example . . . 109
18.5 Systems and Implementation Consequences . . . 109
18.6 Common Misconceptions . . . 110
18.7 Exercises . . . 110
18.8 Further Reading . . . 110

19 Inference and Serving **111**
19.1 Big Picture . . . 111
19.2 Formal Setup and Notation . . . 111
19.3 Main Ideas . . . 111
19.4 Worked Example . . . 111
19.5 Systems and Implementation Consequences . . . 111
19.6 Common Misconceptions . . . 112
19.7 Exercises . . . 112
19.8 Further Reading . . . 112

V Scaling, Evaluation, and Recipes **113**

20 Scaling Laws **115**
20.1 Big Picture . . . 115
20.2 Formal Setup and Notation . . . 115
20.3 Main Ideas . . . 115
20.4 Worked Example . . . 115
20.5 Systems and Implementation Consequences . . . 115
20.6 Common Misconceptions . . . 116
20.7 Exercises . . . 116
20.8 Further Reading . . . 116

21 Model Recipes and Open-Model Case Studies **117**
21.1 Big Picture . . . 117
21.2 Formal Setup and Notation . . . 117
21.3 Main Ideas . . . 117
21.4 Worked Example . . . 117
21.5 Systems and Implementation Consequences . . . 118
21.6 Common Misconceptions . . . 118
21.7 Exercises . . . 118
21.8 Further Reading . . . 118

22 Evaluation **119**
22.1 Big Picture . . . 119
22.2 Formal Setup and Notation . . . 119
22.3 Main Ideas . . . 119
22.4 Worked Example . . . 119
22.5 Systems and Implementation Consequences . . . 119
22.6 Common Misconceptions . . . 120

22.7 Exercises . . . 120
22.8 Further Reading . . . 120

VI Data **121**

23 Data Sources, Crawling, and Cleaning **123**
23.1 Big Picture . . . 123
23.2 Formal Setup and Notation . . . 123
23.3 Main Ideas . . . 123
23.4 Worked Example . . . 123
23.5 Systems and Implementation Consequences . . . 123
23.6 Common Misconceptions . . . 124
23.7 Exercises . . . 124
23.8 Further Reading . . . 124

24 Filtering, Deduplication, and Data Mixing **125**
24.1 Big Picture . . . 125
24.2 Formal Setup and Notation . . . 125
24.3 Main Ideas . . . 125
24.4 Worked Example . . . 125
24.5 Systems and Implementation Consequences . . . 126
24.6 Common Misconceptions . . . 126
24.7 Exercises . . . 126
24.8 Further Reading . . . 126

VII Alignment and Reasoning **127**

25 Instruction Tuning and Preference Tuning **129**
25.1 Big Picture . . . 129
25.2 Formal Setup and Notation . . . 129
25.3 Main Ideas . . . 129
25.4 Worked Example . . . 129
25.5 Systems and Implementation Consequences . . . 130
25.6 Common Misconceptions . . . 130
25.7 Exercises . . . 130
25.8 Further Reading . . . 130

26 RLHF, DPO, GRPO, and Reasoning RL **131**
26.1 Big Picture . . . 131
26.2 Formal Setup and Notation . . . 131
26.3 Main Ideas . . . 131
26.4 Worked Example . . . 131
26.5 Systems and Implementation Consequences . . . 132
26.6 Common Misconceptions . . . 132
26.7 Exercises . . . 132
26.8 Further Reading . . . 132

27 Safety, Reward Hacking, and Deployment **133**
27.1 Big Picture . . . 133
27.2 Formal Setup and Notation . . . 133
27.3 Main Ideas . . . 133
27.4 Worked Example . . . 133
27.5 Systems and Implementation Consequences . . . 134
27.6 Common Misconceptions . . . 134
27.7 Exercises . . . 134
27.8 Further Reading . . . 134

A Proof Techniques and Derivations **137**

Appendices **137**

B CUDA and Triton Cookbook **139**

C Linguistics Quick Reference **141**

D Neuroscience Analogy Index **143**

E Reproducibility and Debugging **145**

References **147**

List of Figures

List of Tables

Part I

Orientation and Prerequisites

Chapter 1

How to Use This Book

This book accompanies Stanford's CS336: Language Modeling from Scratch. It follows the course's arc—from tokenization to alignment—but provides deeper mathematical treatment, more worked examples, and self-contained prerequisite coverage.

1.1 The "From Scratch" Philosophy

When we say "from scratch," we mean it. By the end of this book, you will have seen how to:

- Define the language modeling problem using information theory
- Build a tokenizer from raw text
- Construct a Transformer architecture layer by layer
- Train it efficiently on GPUs with custom kernels
- Scale it using empirical scaling laws
- Curate and filter training data
- Align the model to follow instructions and reason

No black boxes. No "just import this library." Every component is derived, explained, and connected to the underlying mathematics.

1.2 How to Read This Book

1.2.1 Sequential Reading

If you are new to language models, read Part 0 first to solidify prerequisites, then proceed through Parts I–VI in order. This is the recommended path.

1.2.2 "I Know ML" Path

If you are comfortable with probability, linear algebra, and PyTorch, skip Part 0 and start with Chapter 6 (The Language Modeling Problem). Refer back to Part 0 when needed.

1.2.3 Systems Focus

For a systems-oriented reading, focus on Chapters 14–18 after reviewing the Transformer basics in Chapters 9–11.

1.2.4 Assignment Companion

Each CS336 assignment maps to specific chapters:

Assignment	Topic	Chapters
A1	Basics	6–12
A2	Systems	14–17
A3	Scaling	19–21
A4	Data	22–23
A5	Alignment	24–26

1.3 Notation Conventions

We use the following conventions throughout the book:

- **Scalars** are lowercase italic: x, y, α
- **Vectors** are lowercase bold: $\mathbf{x}, \mathbf{y}$
- **Matrices** are uppercase bold: $\mathbf{W}, \mathbf{Q}, \mathbf{K}, \mathbf{V}$
- **Sets** are calligraphic: $\mathcal{V}$ (vocabulary), $\mathcal{D}$ (dataset)
- log always means natural logarithm unless otherwise noted
- Complete notation reference: see the Notation Table on page 5

1.4 Neuroscience Analogies

Throughout this book, you will encounter purple-shaded **Neuroscience Analogy** boxes. These connect machine learning concepts to brain science to build intuition. Every analogy box has three parts:

1. **Where the analogy helps**: the genuine structural or functional similarity

2. **Where the analogy breaks**: the important differences
3. **What intuition to keep**: the one-sentence takeaway

These analogies are thinking tools, not scientific claims about how LLMs work.

1.5 Notation Table

Symbol	**Meaning**
T	Sequence length (context window)
V or $\|\mathcal{V}\|$	Vocabulary size
d or d_{model}	Model / embedding dimension
d_k, d_v	Key and value dimensions per attention head
d_{ff}	Feed-forward hidden dimension
n_h	Number of attention heads
n_{kv}	Number of key-value heads (for GQA)
L	Number of Transformer layers
N	Number of model parameters
D	Dataset size (in tokens)
B	Batch size
η	Learning rate
$\boldsymbol{\theta}$	Parameter vector
$p(x)$	True / empirical distribution
$p_{\boldsymbol{\theta}}(x)$ or $q(x)$	Model distribution
$H(p)$	Entropy
$H(p, q)$	Cross-entropy
$D_{\text{KL}}(p \,\|\, q)$	KL divergence
PPL	Perplexity
$\text{softmax}(\cdot)$	Softmax function (row-wise for matrices)
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	Query, Key, Value matrices
C	Compute budget (FLOPs)
MFU	Model FLOPs Utilization

Chapter 2

Probability, Entropy, and Information Theory

Key Insight

After reading this chapter, you will be able to:

1. Define and compute entropy, cross-entropy, and KL divergence for discrete distributions
2. Derive the connection between maximum likelihood estimation and cross-entropy minimization
3. Explain perplexity and use it to evaluate language models
4. State precisely what a language model is as a probability distribution
5. Use these concepts to motivate why we train language models the way we do

2.1 Big Picture

Every language model is, at its core, a probability distribution over sequences of tokens. When we "train" a model, we are adjusting its parameters so that the probability distribution it represents comes closer to the true distribution of natural language. To make this precise, we need three foundational concepts from information theory:

1. **Entropy** tells us how much uncertainty a distribution has—how "surprised" we should be on average.
2. **Cross-entropy** measures how well one distribution approximates another, and gives us our training loss.

3. **KL divergence** quantifies the "gap" between two distributions, connecting training loss to the information-theoretic ideal.

These are not abstract mathematical curiosities. Cross-entropy is literally the loss function we minimize when training GPT, LLaMA, or any autoregressive language model. Understanding it deeply will make every subsequent chapter more intuitive.

This chapter connects directly to Shannon's 1950 paper on predicting English text [1], which is the intellectual ancestor of all language modeling work.

2.2 Probability Distributions over Tokens

Definition 2.1 (Discrete probability distribution)**.** Let $\mathcal{V} = \{v_1, v_2, \ldots, v_{|\mathcal{V}|}\}$ be a finite set (our **vocabulary**). A **probability distribution** over $\mathcal{V}$ is a function $p : \mathcal{V} \to [0, 1]$ such that

$$\sum_{v \in \mathcal{V}} p(v) = 1. \tag{2.1}$$

We write $p(v)$ for the probability assigned to token v.

In language modeling, $\mathcal{V}$ is our vocabulary of tokens (words, subwords, or characters—see Chapter 7). A language model assigns a probability to each possible next token given a context of previous tokens.

Example 2.1. Suppose our vocabulary is $\mathcal{V} = \{\texttt{the}, \texttt{cat}, \texttt{sat}, \texttt{on}, \texttt{mat}\}$ and after seeing the context "the cat sat on the," our model assigns:

Token v	$p(v \mid$ "the cat sat on the"$)$
`mat`	0.62
`cat`	0.15
`the`	0.08
`sat`	0.10
`on`	0.05

This is a valid probability distribution: all values are in $[0, 1]$ and they sum to 1.00.

2.3 Entropy: Measuring Uncertainty

2.3.1 Definition and Intuition

How "uncertain" or "surprising" is a probability distribution? Shannon's entropy answers this precisely.

Definition 2.2 (Entropy)**.** The **entropy** of a discrete probability distribution p over a finite set $\mathcal{V}$ is

$$H(p) = -\sum_{v \in \mathcal{V}} p(v) \log p(v), \tag{2.2}$$

where log denotes the natural logarithm and we adopt the convention $0 \log 0 = 0$.

Symbol table. $H(p)$ is the entropy (a non-negative real number); $p(v)$ is the probability of token v; the sum is over all tokens in the vocabulary $\mathcal{V}$.

Interpretation. Entropy measures the average "surprise" or "information content" when we draw a token from p. The quantity $-\log p(v)$ is called the **surprisal** of token v: rare events (low $p(v)$) have high surprisal, and common events have low surprisal. Entropy is the expected surprisal.

2.3.2 Key Properties

Proposition 2.1 (Bounds on entropy)**.** *For any distribution p over $\mathcal{V}$ with $|\mathcal{V}| = V$:*

$$0 \leq H(p) \leq \log V. \tag{2.3}$$

- $H(p) = 0$ *if and only if p is a point mass (all probability on one token—zero uncertainty).*
- $H(p) = \log V$ *if and only if p is the uniform distribution (maximum uncertainty).*

Proof. The lower bound follows because $-p(v) \log p(v) \geq 0$ for all v. For the upper bound, we use the fact that log is concave. By Jensen's inequality:

$$H(p) = \mathbb{E}_{v \sim p}[-\log p(v)] \leq -\log\left(\mathbb{E}_{v \sim p}[p(v)]\right) = -\log\left(\sum_{v} p(v)^2\right) \leq \log V, \tag{2.4}$$

with equality when $p(v) = 1/V$ for all v. Alternatively, one can verify this using Lagrange multipliers or the log-sum inequality. □

2.3.3 Worked Example: Computing Entropy

Consider a bigram language model over $\mathcal{V} = \{a, b, c\}$ that predicts the next character with distribution $p(a) = 0.7$, $p(b) = 0.2$, $p(c) = 0.1$:

$$
\begin{aligned}
H(p) &= -[0.7 \log 0.7 + 0.2 \log 0.2 + 0.1 \log 0.1] && (2.5)\\
&= -[0.7 \times (-0.357) + 0.2 \times (-1.609) + 0.1 \times (-2.303)] && (2.6)\\
&= -[-0.250 - 0.322 - 0.230] && (2.7)\\
&= 0.802 \text{ nats.} && (2.8)
\end{aligned}
$$

For comparison, the uniform distribution over 3 tokens has entropy $\log 3 \approx 1.099$ nats. Our distribution has lower entropy because it concentrates most probability on token a—it is more "certain" about what comes next.

Sanity check. The entropy 0.802 is between 0 (point mass) and 1.099 (uniform), as expected.

2.4 Cross-Entropy: Measuring Model Quality

2.4.1 Definition

In language modeling, we have two distributions:

- p: the true distribution of language (unknown, but we have samples from it)
- q (or $p_{\boldsymbol{\theta}}$): our model's predicted distribution

We want to measure how well q approximates p.

Definition 2.3 (Cross-entropy)**.** The **cross-entropy** of distribution q relative to distribution p is

$$H(p, q) = -\sum_{v \in \mathcal{V}} p(v) \log q(v) = \mathbb{E}_{v \sim p}[-\log q(v)]. \tag{2.9}$$

Symbol table. $p(v)$ is the probability of token v under the true distribution; $q(v)$ is the probability under our model; $-\log q(v)$ is the surprisal that our model assigns to token v.

Interpretation. Cross-entropy is the average surprisal when events are drawn from p but we use model q to predict them. If $q = p$, then $H(p, q) = H(p)$—the best we can do. If q diverges from p, cross-entropy increases.

2.4.2 Cross-Entropy as a Training Loss

In practice, we do not know p exactly. Instead, we have a training corpus $\mathcal{D} = (x_1, x_2, \ldots, x_n)$ of n tokens drawn (approximately) from p. The empirical cross-entropy is:

$$\hat{H}(p, q) = -\frac{1}{n} \sum_{i=1}^{n} \log q(x_i \mid x_1, \ldots, x_{i-1}). \tag{2.10}$$

This is exactly the average negative log-likelihood per token—the standard training loss for autoregressive language models such as GPT, LLaMA, and DeepSeek.

Key Insight

When you see a language model training with "cross-entropy loss," it is minimizing exactly Equation (2.10): the average surprisal of the training data under the model. Lower is better.

2.5 KL Divergence: The Gap Between Distributions

Definition 2.4 (KL divergence)**.** The **Kullback–Leibler divergence** from distribution p to distribution q is

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_{v \in \mathcal{V}} p(v) \log \frac{p(v)}{q(v)} = H(p, q) - H(p). \tag{2.11}$$

Interpretation. KL divergence is the "extra cost" (in nats) of using model q instead of the true distribution p. It decomposes cross-entropy into two parts:

$$\underbrace{H(p, q)}_{\text{cross-entropy (what we minimize)}} = \underbrace{H(p)}_{\text{entropy of data (constant)}} + \underbrace{D_{\mathrm{KL}}(p \,\|\, q)}_{\text{gap (what shrinks during training)}}. \tag{2.12}$$

Since $H(p)$ does not depend on the model parameters $\boldsymbol{\theta}$, minimizing cross-entropy $H(p, p_{\boldsymbol{\theta}})$ is equivalent to minimizing $D_{\mathrm{KL}}(p \,\|\, p_{\boldsymbol{\theta}})$.

Proposition 2.2 (Gibbs' inequality)**.** *For any distributions p and q over the same finite set:*

$$D_{\mathrm{KL}}(p \,\|\, q) \geq 0, \tag{2.13}$$

with equality if and only if $p = q$.

Proof. By Jensen's inequality applied to the convex function $-\log$:

$$D_{\mathrm{KL}}(p \,\|\, q) = -\sum_{v} p(v) \log \frac{q(v)}{p(v)} \geq -\log \left(\sum_{v} p(v) \cdot \frac{q(v)}{p(v)} \right) = -\log \left(\sum_{v} q(v) \right) = -\log 1 = 0. \tag{2.14}$$

Equality holds iff $q(v)/p(v)$ is constant for all v with $p(v) > 0$, i.e., $p = q$. □

Common Pitfall

KL divergence is not symmetric: $D_{\mathrm{KL}}(p \,\|\, q) \neq D_{\mathrm{KL}}(q \,\|\, p)$ in general. It is not a metric (distance) in the mathematical sense. In language modeling, we always minimize $D_{\mathrm{KL}}(p \,\|\, p_{\boldsymbol{\theta}})$ (forward KL), not $D_{\mathrm{KL}}(p_{\boldsymbol{\theta}} \,\|\, p)$ (reverse KL). The choice matters: forward KL encourages the model to cover all modes of p (it is "mean-seeking"), while reverse KL encourages the model to concentrate on the highest-probability mode (it is "mode-seeking").

2.6 Maximum Likelihood and Cross-Entropy

We now show that the standard training objective for language models—maximum likelihood estimation—is equivalent to cross-entropy minimization.

Proposition 2.3 (MLE = Cross-entropy minimization)**.** *Given a training corpus $\mathcal{D} = (x_1, \ldots, x_n)$, the maximum likelihood estimate*

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{n} \log p_{\boldsymbol{\theta}}(x_i \mid x_{<i}) \tag{2.15}$$

is equivalent to

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \hat{H}(p_{data}, p_{\boldsymbol{\theta}}). \tag{2.16}$$

Proof. Maximizing $\sum_i \log p_{\boldsymbol{\theta}}(x_i \mid x_{<i})$ is the same as minimizing $-\frac{1}{n}\sum_i \log p_{\boldsymbol{\theta}}(x_i \mid x_{<i})$, which is exactly the empirical cross-entropy $\hat{H}(p_{\text{data}}, p_{\boldsymbol{\theta}})$ from Equation (2.10). The scaling factor $1/n$ does not affect the argmin. □

Key Insight

This is why "next-token prediction" works as a training objective. By predicting the next token as accurately as possible (maximizing likelihood), we are simultaneously minimizing the information-theoretic gap between our model and natural language.

2.7 Perplexity: The Standard Evaluation Metric

Definition 2.5 (Perplexity)**.** The **perplexity** of a model q on a sequence $(x_1, \ldots, x_T)$ is

$$\mathrm{PPL}(q) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log q(x_t \mid x_{<t})\right) = \exp\left(\hat{H}(p, q)\right). \tag{2.17}$$

Interpretation. Perplexity is the exponential of cross-entropy. It can be interpreted as the **effective vocabulary size**: a perplexity of 100 means the model is "as confused" as if it were choosing uniformly among 100 equally likely tokens at each step.

Key relationships.

- A perfect model ($q = p$) achieves PPL $= \exp(H(p))$—the irreducible perplexity determined by the entropy of language itself.
- Lower perplexity = better model.
- Perplexity is the standard metric reported in language modeling papers (e.g., GPT-2, LLaMA).
- If using $\log_2$ instead of ln, we get PPL $= 2^{H_2(p,q)}$ where H_2 is cross-entropy in bits.

2.7.1 Worked Example: Perplexity Calculation

Suppose our model assigns the following probabilities to a 5-token test sequence:

Position t	Token x_t	$q(x_t \mid x_{<t})$
1	`the`	0.20
2	`cat`	0.05
3	`sat`	0.30
4	`on`	0.50
5	`mat`	0.60

Then:

$$\text{avg. NLL} = -\frac{1}{5}\left[\log 0.20 + \log 0.05 + \log 0.30 + \log 0.50 + \log 0.60\right] \tag{2.18}$$

$$= -\frac{1}{5}\left[-1.609 + (-2.996) + (-1.204) + (-0.693) + (-0.511)\right] \tag{2.19}$$

$$= -\frac{1}{5}(-7.013) = 1.403 \tag{2.20}$$

$$\text{PPL} = \exp(1.403) \approx 4.07. \tag{2.21}$$

A perplexity of 4.07 means the model is, on average, as uncertain as if it were choosing uniformly among about 4 tokens—not bad for a vocabulary that might contain tens of thousands of tokens.

Sanity check. The geometric mean of the per-token probabilities is $(0.20 \times 0.05 \times 0.30 \times 0.50 \times 0.60)^{1/5} \approx 0.245$, and $1/0.245 \approx 4.07$. This confirms: perplexity is the reciprocal of the geometric mean probability.

2.8 Shannon's Game and the Entropy Rate of English

In 1950, Claude Shannon asked: how much information does each letter in English carry [1]? He devised a guessing game where a human predicts the next character in a text, and used the results to estimate the entropy rate of English at approximately 1.0–1.5 bits per character.

This experiment is the direct ancestor of modern language model evaluation:

- Shannon's human predictor $\longrightarrow$ our neural language model $p_{\boldsymbol{\theta}}$
- Shannon's guessing accuracy $\longrightarrow$ our model's perplexity
- Shannon's entropy bound $\longrightarrow$ the irreducible loss floor

Modern language models like GPT-4 approach or surpass human-level prediction accuracy on many text domains, suggesting they have learned much of the statistical structure of language. But as we will see in later chapters, statistical prediction is not the same as understanding.

Neuroscience Analogy: Predictive Coding in the Brain

Where the analogy helps. The brain constantly generates predictions about upcoming sensory input. When the actual input matches the prediction, neural activity is low (low surprise). When the input is unexpected, prediction error signals propagate up the cortical hierarchy. This is strikingly parallel to how a language model computes surprisal $-\log p_{\boldsymbol{\theta}}(x_t \mid x_{<t})$: expected tokens incur low loss, surprising tokens incur high loss.

Where the analogy breaks. The brain's predictive coding operates across multiple modalities (vision, touch, proprioception) simultaneously, uses lateral and feedback connections, and updates predictions in real time with no distinct "training phase." A Transformer language model processes text only, uses feedforward computation (with residual connections, not true recurrence), and has a sharp divide between training and inference.

What intuition to keep. Think of next-token prediction loss as a measure of the model's "surprise"—exactly like prediction error signals in the brain. Training reduces this surprise by learning the statistical patterns of language. But the model's "prediction" is purely statistical, not a theory of the world.

2.9 Common Misconceptions

1. **"Lower cross-entropy always means a better model."** Only when comparing models on the same data with the same tokenization. Different tokenizations change the token count T, making cross-entropy values incomparable. Always compare with matched tokenization or use bits-per-byte.

2. **"KL divergence is a distance metric."** It is not: $D_{\mathrm{KL}}(p \,\|\, q) \neq D_{\mathrm{KL}}(q \,\|\, p)$ in general, and it does not satisfy the triangle inequality. Treat it as a directed measure of information loss.

3. **"Perplexity of 1 means a perfect model."** A perplexity of 1 would mean the model predicts every token with probability 1—this is impossible for natural language, which has genuine uncertainty. A "perfect" model would achieve $\mathrm{PPL} = \exp(H(p))$, which is greater than 1.

4. **"Cross-entropy loss is just a convention; we could use MSE."** Cross-entropy is the uniquely correct loss for maximum likelihood training of categorical distributions. Using MSE would correspond to a different (and generally worse) probabilistic assumption about the output.

5. **"Shannon showed that English has about 1 bit of entropy per character, so LLMs are almost done."** Shannon estimated the *entropy rate per character*. Modern subword models operate over much larger token units. The entropy rate of language depends on the unit of measurement, and there is still substantial room for improvement, especially for rare and specialized language.

2.10 Exercises

Warmup

1. Compute the entropy of the distribution $p(a) = 0.5, p(b) = 0.25, p(c) = 0.25$ in both nats and bits.

2. A model assigns probability 0.1 to a token that actually appeared. What is the surprisal in nats? In bits?

3. If a model has perplexity 50 on a test set, what is its average cross-entropy loss per token (in nats)?

Derivation

1. Prove that among all distributions over $\{1, 2, \ldots, V\}$, the uniform distribution uniquely maximizes entropy. (Hint: use Lagrange multipliers or the log-sum inequality.)

2. Show that $D_{\mathrm{KL}}(p \,\|\, q) \geq 0$ using the log-sum inequality instead of Jensen's inequality.

3. Derive the relationship between perplexity and the geometric mean of per-token probabilities: $\mathrm{PPL} = \left(\prod_{t=1}^{T} q(x_t \mid x_{<t})\right)^{-1/T}$.

Implementation

1. [**Prepares for CS336 A1**] Write a Python function `compute_perplexity(log_probs: list[float]) -> float` that takes a list of log-probabilities $\log q(x_t \mid x_{<t})$ and returns the perplexity of the sequence. Test it on the worked example from Section 2.7.

2. Write a function that takes two discrete probability distributions (as NumPy arrays) and computes their cross-entropy and KL divergence. Verify that $D_{\mathrm{KL}}(p \,\|\, q) = H(p, q) - H(p)$.

Open-Ended

1. Shannon's 1950 experiment used human predictors. How would you design a modern version of this experiment using a language model? What would change if you used GPT-4 instead of a human? What aspects of language might a human still predict better than a model?

2.11 Further Reading

- "Prediction and Entropy of Printed English" [1]: The foundational paper on predicting English text. Introduces the entropy rate experiments that motivate all language modeling work. Remarkably readable for a 1950 paper.

- Cover & Thomas, *Elements of Information Theory*, Chapter 2: A rigorous treatment of entropy, relative entropy (KL divergence), and mutual information. The standard reference for information theory.

- MacKay, *Information Theory, Inference, and Learning Algorithms*, Chapters 2–4: A more intuitive and example-rich treatment of the same material, freely available online.

- "A Neural Probabilistic Language Model" [2]: The paper that introduced neural language models. Uses cross-entropy training as described in this chapter. Reading it shows how the information-theoretic framework connects to neural architectures.

Chapter 3

Linear Algebra and Matrix Calculus

Key Insight

After reading this chapter, you will be able to:

1. Perform block matrix multiplication and interpret matrices as collections of vectors.
2. Understand Einstein summation notation (einsum) as a universal language for tensor contractions.
3. Compute the gradients of scalar-valued functions with respect to vectors and matrices.
4. Derive the backdrop equations for a linear layer and a simple MLP.
5. Understand the dimensional transformations induced by the self-attention mechanism.

3.1 Big Picture

Language models are essentially massive sequences of matrix multiplications punctuated by non-linearities. Consequently, fluency in linear algebra is non-negotiable.

However, the kind of linear algebra used in machine learning is heavily skewed. You will rarely need to compute a determinant, find a matrix inverse, or diagnose linear independence. Instead, ML linear algebra focuses almost entirely on **tensor reshaping**, **block multiplication**, and **matrix calculus** for backpropagation.

If probability theory (Chapter 1) tells us *what* to compute (likelihood), linear algebra tells us *how* we represent and transform the data, and matrix calculus tells us *how* to optimize the parameters to get there.

3.2 Formal Setup and Notation

We work over the field of real numbers, $\mathbb{R}$.

- A **scalar** is a single number $x \in \mathbb{R}$.
- A **vector** is a 1D array of numbers $\mathbf{x} \in \mathbb{R}^n$. By convention, all vectors are **column vectors**. A row vector is denoted as $\mathbf{x}^\mathsf{T} \in \mathbb{R}^{1\times n}$.
- A **matrix** is a 2D array $\mathbf{W} \in \mathbb{R}^{m\times n}$ with m rows and n columns.
- A **tensor** is a generalization to arbitrary dimensions. An N-dimensional tensor is $\mathbf{T} \in \mathbb{R}^{d_1\times d_2\times\cdots\times d_N}$.

3.3 Matrix Multiplication Views

Given $\mathbf{W} \in \mathbb{R}^{m\times n}$ and $\mathbf{x} \in \mathbb{R}^n$, the product $\mathbf{y} = \mathbf{W}\mathbf{x}$ is a vector in $\mathbb{R}^m$. Instead of computing cell-by-cell ($y_i = \sum_j W_{ij}x_j$), you should cultivate three geometric views:

1. The Column Space View. $\mathbf{y}$ is a linear combination of the columns of $\mathbf{W}$, weighted by the elements of $\mathbf{x}$. If $\mathbf{W} = [\mathbf{c}_1 \mid \mathbf{c}_2 \mid \cdots \mid \mathbf{c}_n]$, then $\mathbf{y} = x_1\mathbf{c}_1 + x_2\mathbf{c}_2 + \cdots + x_n\mathbf{c}_n$.

2. The Row Space View. Every element y_i is the dot product of the i-th row of $\mathbf{W}$ with $\mathbf{x}$.

3. The Dimensionality Transformation. $\mathbf{W}$ acts as a function mapping an n-dimensional space to an m-dimensional space.

3.3.1 Block Matrix Multiplication

In language models, we process sequences of length T, where each token is a vector of dimension d. We stack the T column vectors *horizontally* (Wait, no! In ML libraries, we stack them vertically. We will use the standard ML convention: the data matrix $\mathbf{X}$ has shape $T \times d$, so each *row* is a token).

If $\mathbf{X} \in \mathbb{R}^{T\times d}$ is our sequence, and we want to project it to a new dimension k using weights $\mathbf{W} \in \mathbb{R}^{d\times k}$, we compute:

$$\mathbf{Y} = \mathbf{X}\mathbf{W} \in \mathbb{R}^{T\times k}. \tag{3.1}$$

Because matrix multiplication distributes over rows, the t-th row of $\mathbf{Y}$ depends *only* on the t-th row of $\mathbf{X}$. This means $\mathbf{X}\mathbf{W}$ applies the exact same linear transformation to every token in the sequence independently and in parallel.

3.4 Einstein Summation Notation (Einsum)

Standard matrix notation ($\mathbf{AB}^\mathsf{T}$) completely collapses when dealing with 3D or 4D tensors (like a batch of sequences of multi-head attention components). We rely on **Einstein summation notation** to write unambiguous tensor contractions.

The rule of Einsum is simple: **repeated indices are summed over.**

- Dot product of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$: `i, i ->` $\implies c = \sum_i x_i y_i$
- Matrix-vector multiplication $\mathbf{Ax}$: `ij, j -> i` $\implies y_i = \sum_j A_{ij} x_j$
- Matrix multiplication $\mathbf{AB}$: `ij, jk -> ik` $\implies C_{ik} = \sum_j A_{ij} B_{jk}$
- Batched matrix multiplication: `bij, bjk -> bik` $\implies C_{bik} = \sum_j A_{bij} B_{bjk}$
- Attention scores: $\mathbf{Q} \in \mathbb{R}^{B \times h \times T \times d_k}, \mathbf{K} \in \mathbb{R}^{B \times h \times T \times d_k}$. We want the dot product of queries and keys over d_k, leaving a $T \times T$ matrix per batch per head. `b h t d, b h s d -> b h t s`

Every complex reshape/transpose/bmm in PyTorch can be replaced with a single `torch.einsum`.

3.5 Matrix Calculus Basics

We train neural networks by optimizing a scalar loss function $\mathcal{L} \in \mathbb{R}$ with respect to millions of parameters $\boldsymbol{\theta}$. We need to compute gradients of scalars with respect to vectors and matrices.

3.5.1 The Gradient of a Scalar w.r.t a Vector

The gradient of a scalar $\mathcal{L}$ with respect to a vector $\mathbf{x} \in \mathbb{R}^n$ is a vector of the same shape $\mathbb{R}^n$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial x_1} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial x_n} \end{bmatrix} \tag{3.2}$$

3.5.2 The Chain Rule via Jacobians

Suppose we have a composition $\mathbf{x} \xrightarrow{f} \mathbf{y} \xrightarrow{g} \mathcal{L}$, where $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{y} \in \mathbb{R}^m$, and $\mathcal{L} \in \mathbb{R}$. By the multivariate chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^\mathsf{T} \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \tag{3.3}$$

where $\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \in \mathbb{R}^{m \times n}$ is the **Jacobian matrix** containing partial derivatives $\frac{\partial y_i}{\partial x_j}$.

3.5.3 Key Matrix Calculus Rules

You must memorize these two rules for linear layers: 1. If $\mathbf{y} = \mathbf{W}\mathbf{x}$, then the gradient w.r.t $\mathbf{x}$ is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \tag{3.4}$$

2. The gradient w.r.t to the weight matrix $\mathbf{W}$ is the outer product of the incoming gradient and the input:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \mathbf{x}^\top \tag{3.5}$$

For batched inputs $\mathbf{Y} = \mathbf{X}\mathbf{W}^\top$ (where $\mathbf{X} \in \mathbb{R}^{B \times d}$, $\mathbf{W} \in \mathbb{R}^{h \times d}$), the gradient w.r.t weights averages over the batch:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \right)^\top \mathbf{X} \tag{3.6}$$

3.6 Worked Example: Backprop through a Linear Layer

Let $\mathbf{x} \in \mathbb{R}^d$ be an input token. We project it through a linear layer with no bias: $\mathbf{z} = \mathbf{W}\mathbf{x}$ where $\mathbf{W} \in \mathbb{R}^{k \times d}$, then apply a scalar loss $\mathcal{L} = ||\mathbf{z}||_2^2 = \mathbf{z}^\top \mathbf{z}$.

We want to find the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$.

1. First, find $\frac{\partial \mathcal{L}}{\partial \mathbf{z}}$. Since $\mathcal{L} = \sum_i z_i^2$, the derivative w.r.t z_i is $2z_i$. Thus $\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = 2\mathbf{z}$. 2. Next, apply the matrix calculus rule for weights:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \mathbf{x}^\top = 2\mathbf{z}\mathbf{x}^\top \in \mathbb{R}^{k \times d} \tag{3.7}$$

Sanity check: Does the shape of the gradient match the shape of the parameter? $\mathbf{W}$ is $k \times d$. $\mathbf{z}$ is $k \times 1$, $\mathbf{x}^\top$ is $1 \times d$. Their outer product is $k \times d$. The shapes match exactly. Always check gradient shapes!

3.7 Systems and Implementation Consequences

Why not just write ‘for‘ loops instead of using matrices? Modern hardware (GPUs, TPUs) contains specialized silicon (Tensor Cores / Matrix Multiply Units) designed explicitly to perform $D = A \times B + C$ on small blocks of data simultaneously.

A single matrix multiplication $\mathbf{A}_{M \times K} \times \mathbf{B}_{K \times N}$ requires $2 \cdot M \cdot N \cdot K$ floating point operations (FLOPs). A modern NVIDIA H100 can execute nearly a *quadrillion* FLOPs per second, provided the matrices are large enough to keep thousands of streaming multiprocessors fed. If your matrices are too small, or you try to loop over vectors sequentially, bandwidth outpaces compute, and you utilize a fraction of a percent of the GPU.

Neuroscience Analogy: Synaptic Weights vs. Matrix Multiplication

Where the analogy helps. If a population of neurons $\mathbf{x}$ sends signals to a downstream population $\mathbf{y}$, the synaptic connections form a matrix $\mathbf{W}$. The activity of a downstream neuron y_i is literally the weighted sum (dot product) of the incoming activities x_j, exactly modeling $y_i = \sum_j W_{ij} x_j$.

Where the analogy breaks. Biological synapses cannot easily support the "transpose" operation $\mathbf{W}^\mathsf{T}$ required for backpropagation. Furthermore, dense matrix multiplication implies every neuron is connected to every other neuron, whereas biological networks are extremely sparse (often $< 1\%$ connectivity).

What intuition to keep. Matrix multiplication maps one distributed concept representation into another via a dense web of overlapping connections, allowing for massively parallel computation.

3.8 Common Misconceptions

1. **"I need to compute the inverse of the matrix to go backward."** Matrix inversion is virtually never used in deep learning. Gradients use the transpose $\mathbf{W}^\mathsf{T}$, not the inverse $\mathbf{W}^{-1}$.

2. **"Backpropagation is a mysterious calculus trick."** It is literally just the multivariable chain rule applied to sequences of matrix multiplications, caching intermediate results (activations) to avoid redundant computation.

3. **"Einsum is slower than dedicated PyTorch functions."** Historically true, but modern compiler stacks (like `torch.compile`) optimize `einsum` expressions into highly efficient kernel calls perfectly equivalent to dedicated `bmm` operations.

3.9 Exercises

Warmup

1. Write the Einstein summation string for extracting the diagonal of a matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$.

2. Let $\mathbf{A}$ be a 4×3 matrix and $\mathbf{B}$ be a 3×5 matrix. What is the shape of $(\mathbf{AB})^\mathsf{T}$?

Derivation

1. Derive the gradient of the scalar function $f(\mathbf{x}) = \mathbf{x}^\mathsf{T}\mathbf{A}\mathbf{x}$ with respect to $\mathbf{x}$. (Assume $\mathbf{A}$ is a symmetric matrix).

Implementation

1. [**Prepares for CS336 A1**] Open a Python REPL. Create two random 3D PyTorch tensors $\mathbf{A}$ of shape (B, N, d) and $\mathbf{B}$ of shape (B, M, d). Write a single `torch.einsum` command that computes the dot product of every d-dimensional vector in $\mathbf{A}$ with every d-dimensional vector in $\mathbf{B}$, resulting in a tensor of shape (B, N, M).

Open-Ended

1. Why do you think deep learning frameworks universally represent gradients as having the exact same shape as the parameter tensor, even though mathematically the derivative of a loss scalar w.r.t a matrix should technically be a covector (or 1-form)? What practical engineering benefit does this equality of shapes provide?

3.10 Further Reading

- *The Matrix Calculus You Need For Deep Learning* (Terence Parr and Jeremy Howard). An excellent, approachable paper that demystifies derivatives with respect to vectors and matrices without resorting to tedious index hunting.
- *Linear Algebra Done Right* (Sheldon Axler). For a rigorous foundation building up to vector spaces and linear maps, focusing on geometric intuition rather than rote determinants calculation.

Chapter 4

PyTorch and Tensor Computation

Key Insight

After reading this chapter, you will be able to:

1. Explain how PyTorch represents tensors in memory, distinguishing between contiguous and strided layouts.
2. Understand broadcasting semantics and use them to avoid unnecessary memory allocation.
3. Trace the computational graph in PyTorch's autograd engine.
4. Correctly detach tensors, use `torch.no_grad()`, and manage memory effectively.

4.1 Big Picture

Mathematical formulas in papers must eventually hit silicon. PyTorch is the dominant framework in modern AI research because it strikes an elegant balance between pythonic imperativity and high-performance compiled execution.

However, writing *correct* PyTorch code is not the same as writing *fast* PyTorch code. To build language models from scratch, you cannot just memorize API calls (`self.linear = nn.Linear(...)`). You must understand what happens underneath the hood: how numbers are sequenced linearly in RAM, how strides allow constant-time reshaping, how broadcasting avoids massive copying, and how the autograd engine secretly builds a directed acyclic graph behind your back.

This chapter bridges the gap between the matrix math of Chapter 2 and the hardware realities we will face in Chapter 14.

4.2 Formal Setup and Notation

A **tensor** conceptually is an n-dimensional array. In hardware memory, there are no dimensions—memory is a strict 1D sequence of bytes.

PyTorch maps an n-dimensional tensor indices $(i_0, i_1, \ldots, i_{n-1})$ to a flat 1D memory index k using a **shape** array $S = (s_0, \ldots, s_{n-1})$ and a **stride** array $D = (d_0, \ldots, d_{n-1})$:

$$k = \text{offset} + \sum_{j=0}^{n-1} i_j \cdot d_j \tag{4.1}$$

A tensor is **contiguous** if its items are laid out back-to-back in memory in a strict row-major order (similar to C arrays). Mathematically, a contiguous tensor satisfies $d_j = \prod_{k=j+1}^{n-1} s_k$, with $d_{n-1} = 1$.

4.3 Strides, Views, and Contiguity

Operations like `.view()`, `.transpose()`, and `.squeeze()` do *not* move data in memory. They simply create a new tensor object that points to the same underlying storage, but with modified *shape* and *stride* metadata. This makes them $O(1)$ operations, completely free in terms of compute.

For instance, if you have a matrix $\mathbf{X}$ of shape $(100, 20)$ that is contiguous, its strides are $(20, 1)$. If you call $\mathbf{Y} = \mathbf{X}.\mathbf{transpose}(\mathbf{0}, \mathbf{1})$, $\mathbf{Y}$ has shape $(20, 100)$ and strides $(1, 20)$.

Because $\mathbf{Y}$'s strides jump around in memory, $\mathbf{Y}$ is no longer contiguous. Certain low-level CUDA kernels (like some matrix multiplications or flattening layers) require inputs to be contiguous. If you call `.contiguous()` on $\mathbf{Y}$, PyTorch will be forced to allocate new memory and physically copy the data row-by-row into a tight, back-to-back memory layout, which is expensive.

4.4 Broadcasting

Broadcasting is a mechanism that allows PyTorch to perform element-wise arithmetic between tensors of different shapes, without physically replicating the smaller tensor.

The Broadcasting Rule: Two dimensions are compatible if they are equal, or if one of them is 1. PyTorch aligns shapes from the trailing (right-most) dimension to the left.

Suppose you have an activation tensor $\mathbf{A}$ of shape (B, T, d) and a bias vector $\mathbf{b}$ of shape (d).

- Align right: `A`: (B, T, d)
- Align right: `b`: $(\text{None}, \text{None}, d)$

Since d matches d, and the leading dimensions are missing in $\mathbf{b}$, PyTorch treats $\mathbf{b}$ as having shape $(1, 1, d)$ and virtually expands it to (B, T, d) with a stride of 0 along the expanded dimensions. $\mathbf{A} + \mathbf{b}$ computes seamlessly without using extra memory.

4.5 Autograd: Automatic Differentiation

Every time you perform an operation on tensors that have `requires_grad=True`, PyTorch constructs a dynamic Directed Acyclic Graph (DAG).

- **Leaves**: Your raw parameters $(\mathbf{W}, \mathbf{b})$.
- **Edges**: The mathematical operations (`AddBackward`, `MmBackward`).
- **Root**: The scalar loss $\mathcal{L}$.

When you call `loss.backward()`, PyTorch traverses the DAG from the root down to the leaves, applying the chain rule (Chapter 2) at every node. It accumulates the gradients into the `.grad` attribute of the leaf tensors.

Because the graph consumes memory to store intermediate activations needed for the backward pass, tracking history when you only want to do inference will cause massive memory leaks.

Common Pitfall

Inference trap: Always wrap evaluation code tightly in a `with torch.no_grad():` context block. This completely disables the dynamic DAG construction, saving immense amounts of RAM and compute.

4.6 Worked Example: The Gradient Accumulation Pattern

When training an LLM, the batch size physically capable of fitting in GPU VRAM is often smaller than the batch size required for stable convergence. We solve this using gradient accumulation:

```python
model.zero_grad()  # 1. Clear old gradients
desired_batch = 64
micro_batch = 8
accumulation_steps = desired_batch // micro_batch

for step, (x, y) in enumerate(dataloader):
    # Forward pass builds the DAG
    logits = model(x)
    loss = compute_loss(logits, y)

    # Scale loss because gradients sum over backward passes
```

```
    loss = loss / accumulation_steps

    # Backward pass traverses the DAG, adds to .grad, frees the graph
    loss.backward()

    # Only update parameters every N steps
    if (step + 1) % accumulation_steps == 0:
        optimizer.step()
        model.zero_grad()
```

Gradient Accumulation

Sanity Check: Why scale the loss by `accumulation_steps`? Because the loss function (like CrossEntropy) averages over the batch. If you process 8 micro-batches and add their gradients, you effectively summed the loss instead of averaged it over all 64 elements. Scaling by $\frac{1}{8}$ perfectly recreates the exact mathematics of processing a batch of 64 globally.

4.7 Systems and Implementation Consequences

A huge source of slow training is CPU-GPU sync. If you call `loss.item()` or `print(tensor)`, the CPU must immediately halt and wait for the GPU to finish all queued computation just to fetch that single number. This breaks the asynchronous execution pipeline. Keep printing and `.item()` fetching strictly to boundaries, such as once every 100 log steps.

Neuroscience Analogy: Synaptic Plasticity vs. Autograd

Where the analogy helps. Both frameworks involve a "forward" pass (sensory perception / action) followed by an evaluation (reward/error), which triggers a weight update.

Where the analogy breaks. Autograd requires a perfect global memory of exact neural activations from the forward pass to compute gradients (stored in the DAG). Brains lack this unphysical mechanism; biological synapses update using strictly local information (like spike timing between two adjacent neurons) and diffuse global neuromodulators (like dopamine), lacking coordinate-perfect retrograde tracking.

What intuition to keep. Autograd is an engineering hack—a brilliant accounting system—that forces the network to learn. It guarantees exact optimization mathematics, at the cost of requiring immense amounts of memory to store the past.

4.8 Common Misconceptions

1. **"`.reshape()` and `.view()` are exactly the same thing."** `.view()` is strict: it will crash if the tensor is not contiguous because it promises not to copy data. `.reshape()` acts like `.view()` if possible, but will secretly copy the data and make it contiguous if it has to. Know which one you are using!

2. **"Gradients are overwritten on `loss.backward()`."** False! Gradients are *accumulated* (added) into the `.grad` attribute. If you don't call `optimizer.zero_grad()`, your next step will add the new gradients to the old ones, destroying your optimization trajectory.

4.9 Exercises

Warmup

1. You have a tensor $\mathbf{X}$ of shape $(10, 50)$ with strides $(50, 1)$. You slice it: $\mathbf{X}[:, 0]$. What is the shape and stride of the new 1D tensor?

2. Why does `loss.backward()` throw a runtime error if you call it twice in a row without running a new forward pass?

Derivation

1. If a contiguous tensor has shape (B, T, d), state the complete stride array in terms of B, T, and d.

Implementation

1. [**Prepares for CS336 A1**] Write a function `rope(x)` that takes a tensor `x` of shape `(B, T, d)` and correctly applies broadcasting to add a learned positional encoding `bias` of shape `(T, d)` to it without using loops or `.expand()`.

Open-Ended

1. Given how Autograd retains the computation graph, hypothesize why Transformer architectures require so much GPU memory during training compared to inference, even for tiny batch sizes. What exactly is taking up that space?

4.10 Further Reading

- *PyTorch Autograd Mechanics* (PyTorch Docs): The canonical, definitive guide on how autograd interacts with memory and backpropagation.

- *ezyang's blog on PyTorch Internals*: Deep dives into the Tensor → Storage offset mechanism by a core PyTorch developer.

Chapter 5

CUDA and GPU Execution

Key Insight

After reading this chapter, you will be able to:

1. Understand the fundamental architectural differences between CPUs and GPUs.
2. Explain the CUDA programming model: grids, blocks, and threads.
3. Differentiate between High Bandwidth Memory (HBM) and SRAM (Shared Memory), and explain memory bandwidth bottlenecks.
4. Understand the concept of arithmetic intensity.

5.1 Big Picture

Language models do not run on floating, abstract mathematical planes; they run on brutally physical silicon grids. If Chapter 3 explained how PyTorch organizes data in RAM, this chapter explains why we need specialized hardware to compute on it.

A CPU is a sports car: it is incredibly fast at executing a complex, sequential set of instructions to get one passenger to a destination quickly. A GPU is a freight train: it is slow to get moving and inflexible in its route, but it can transport ten thousand passengers simultaneously.

Because modern neural network arithmetic primarily consists of massive, independent matrix dot-products, we exclusively train them on GPUs. However, squeezing performance out of a GPU requires understanding its memory hierarchy. The hardest part of scaling language models is not doing the math, but moving the data exactly to the right place at the right time so the math units don't idle.

This chapter introduces the fundamental concepts of GPU execution, preparing you for the deep systems analysis in Chapters 14–17.

5.2 Formal Setup and Notation

We abstract the GPU as two separate domains: **Compute** and **Memory**.

- **FLOPs (Compute)**: Floating Point Operations per second. This is how fast the GPU can multiply and add numbers.
- **Bandwidth (Memory)**: Bytes per second. This is how fast data can be moved from the storage to the compute units.

A **kernel** is a small program (usually written in CUDA C++ or Triton) that is compiled by the CPU but executed natively on the GPU. PyTorch hides these kernels behind clean API calls (e.g., `tensor.matmul()`).

5.3 The Compute Hierarchy: Grids, Blocks, Threads

When you launch a kernel on a GPU, you cannot just tell it "process this matrix." You must define an execution grid.

1. Thread (The Engine) A thread is the smallest unit of execution. It possesses its own registers. A single thread might be responsible for computing exactly one element of an output matrix.

2. Block (The Train Car) Threads are grouped into **thread blocks** (e.g., $16 \times 16 = 256$ threads). All threads in a block execute simultaneously on the same physical Streaming Multiprocessor (SM). Crucially, threads within the same block can communicate instantly and share a pool of ultrafast memory.

3. Grid (The Train) Blocks are grouped into a **grid**. If we want to compute an output matrix of size $M \times N$, and our blocks are 16×16, we need a grid of size $\lceil M/16 \rceil \times \lceil N/16 \rceil$ blocks.

The GPU scheduler dictates which block runs on which SM. You have no guarantee of the order in which blocks execute.

5.4 The Memory Hierarchy and the Bandwidth Wall

The single most important concept in modern AI engineering is the memory hierarchy.

1. High Bandwidth Memory (HBM | VRAM) The 80GB of memory advertised on an NVIDIA A100 is HBM. It is massive, but relative to computation, it is extremely *slow*. Standard PyTorch tensors live here. Fetching data from HBM takes hundreds of clock cycles.

2. SRAM (Shared Memory / L1 Cache) Each SM has a tiny sliver (e.g., 192 KB) of static RAM. It is physically located right next to the math cores. Accessing it takes only a few clock cycles.

3. Registers The ultra-fast memory physically inside the thread.

5.4.1 The Bandwidth Bottleneck

Suppose an A100 GPU can do 3×10^{14} math operations per second (300 TFLOPs), but its HBM bandwidth is only 2×10^{12} bytes per second (2 TB/s).

If you want to add two vectors $\mathbf{z} = \mathbf{x} + \mathbf{y}$, you must load two floats (8 bytes total) and write one float (4 bytes). Total movement: 12 bytes. Total compute: 1 FLOP. The mathematical hardware will finish the addition instantly, then sit completely idle waiting for the data to arrive from HBM. The operation is strictly **memory-bound**.

Because math is fast and memory is slow, the secret to fast code is **kernel fusion**. Keep data in SRAM as long as possible, doing as much math as you can on it, before writing it back to HBM.

5.5 Arithmetic Intensity

To quantify the balance between compute and memory, we define **Arithmetic Intensity** (I):

$$I = \frac{\text{FLOPs performed}}{\text{Bytes accessed from HBM}} \tag{5.1}$$

Vector addition has $I \approx 1/12$. Matrix multiplication $\mathbf{A}_{N\times N} \times \mathbf{B}_{N\times N}$ requires N^3 operations, but only $2N^2$ bytes of data. Therefore, $I \approx \frac{N}{2}$. As matrices get large, I passes the threshold where the GPU spends all its time doing math rather than waiting for memory. This is called being **compute-bound**.

We want our neural network layers to be compute-bound.

5.6 Worked Example: Kernel Overhead

When you write this in PyTorch:

```
y = x * 2.0
z = y + 1.0
```

What actually happens on the GPU? 1. PyTorch launches the `multiply` kernel. 2. The GPU reads `x` from HBM, multiplies by 2, and writes intermediate `y` to HBM. 3. PyTorch launches the `add` kernel. 4. The GPU reads `y` from HBM, adds 1.0, and writes `z` to HBM.

Total HBM accesses: 1 read of `x`, 1 write of `y`, 1 read of `y`, 1 write of `z`.

If we fused this into a single kernel (`z = x * 2.0 + 1.0`), the GPU would read `x` *once*, do both math operations in registers, and write `z` *once*. Kernel fusion cuts the memory traffic in half. This principle is what drives technologies like `torch.compile` and FlashAttention.

5.7 Systems and Implementation Consequences

Because of the blocking architecture, data shapes dramatically impact performance. If your matrix has a dimension of 17, and the hardware optimally processes blocks of 16, the GPU must fire up a whole new block just to handle that 1 remaining row. The other 15 threads in that block will mask out their computation (do nothing), wasting power and compute.

Always pad your sequence lengths, vocabulary sizes, and hidden dimensions to be powers of 2, or at the very least, multiples of 8 or 16. A dimension of 512 trains drastically faster and smoother than a dimension of 511.

Neuroscience Analogy: Grey Matter vs. White Matter

Where the analogy helps. In the central nervous system, grey matter consists of neuronal cell bodies and processing hubs (analogous to SRAM and SMs doing compute). White matter consists of myelinated axons acting as long-range communication highways (analogous to HBM fetching and PCIE bandwidth).

Where the analogy breaks. The brain physically intertwines memory and compute—the synapse stores the weight *and* performs the multiplication simultaneously. A GPU physically separates the storage (HBM) from the math core (ALU), introducing the "von Neumann bottleneck" that forces data to commute.

What intuition to keep. Just as an organism is severely limited if its neural tracts (white matter) are too narrow to feed information to the cortex quickly enough, a GPU is useless if its memory bandwidth cannot feed its teraflop compute cores. Data movement is the ultimate bottleneck.

5.8 Common Misconceptions

1. **"GPU programming is just C++ with weird syntax."** The syntax is trivial. The difficulty is engineering the hierarchy so that threads cooperate, avoid bank conflicts in SRAM, hide latency via asynchronous fetching, and maximize arithmetic intensity.

2. **"Doing more math slows the model down."** Because models are frequently memory-bound, you can often compute complex polynomial approximations (like GELU) in registers "for free" because the latency is entirely hidden underneath the time it takes to fetch the next block of memory. Math is cheap; memory is expensive.

5.9 Exercises

Warmup

1. Explain why the Softmax function requires two massive sweeps over the input vector, and why this makes it an extremely memory-bound operation.

Derivation

1. Define the **ridge point** of a GPU as the arithmetic intensity where the hardware transitions from memory-bound to compute-bound. If a GPU has C TFLOPs of compute and B TB/s of bandwidth, derive a formula for the ridge point.

Implementation

1. [**Prepares for CS336 A2**] Open a Python environment with PyTorch. Write a script that compares the execution time of `y = torch.exp(x) + torch.sin(x)` against a compiled version generated by `torch.compile`.

Open-Ended

1. In 2020, Apple introduced "Unified Memory" where the CPU and GPU physically share the same pool of RAM. Given what you know about High Bandwidth Memory vs. System RAM latency, what are the advantages and drawbacks for training LLMs on an architecture without discrete VRAM?

5.10 Further Reading

- *CUDA C++ Programming Guide* (NVIDIA). The definitive documentation on grids, blocks, and memory hierarchy.
- *Making Deep Learning Go Brrrr From First Principles* (Horace He). An essential, accessible blog post on arithmetic intensity and the difference between memory-bound and compute-bound kernels in PyTorch.

Chapter 6

Minimal Linguistics Background

Key Insight

After reading this chapter, you will be able to:

1. Distinguish between syntax, semantics, and pragmatics.
2. Understand morphology and explain why agglutinative languages challenge fixed-vocabulary systems.
3. Explain Zipf's Law and its mathematical implications for rare-word modeling.
4. Understand why formal grammars (like Context-Free Grammars) are insufficient for natural language.

6.1 Big Picture

Language modeling research is a branch of Computer Science, but our data structure is a product of human evolution and culture. While modern Deep Learning emphasizes throwing scale at raw data without hand-coded linguistic rules (a philosophy popularized as "the bitter lesson"), ignoring linguistics entirely is an engineering mistake.

Why? Because the statistical properties of natural language heavily dictate our tokenization schemes, our scaling laws, and our evaluation metrics. If you understand Zipf's Law, you understand why large vocabularies are mathematically mandatory. If you understand morphology, you understand why German and Turkish break bad tokenizers.

This chapter is not a crash course in Chomsky; it provides the absolute minimum descriptive linguistics required to debug, evaluate, and scale language models across English and multilingual corpora.

6.2 Formal Setup and Notation

Linguists analyze language across a hierarchy of abstraction:

- **Phonetics/Phonology**: The physical sounds and sound systems of speech. (Largely irrelevant for text-based LLMs).
- **Morphology**: The internal structure of words (roots, prefixes, suffixes).
- **Syntax**: The rules governing the structural arrangement of words into phrases and sentences.
- **Semantics**: The literal meaning of words, phrases, and sentences.
- **Pragmatics**: Meaning derived from context, intent, and social convention.

6.3 Morphology and Typology

Morphemes are the smallest units of meaning in a language. The word "unbelievable" is composed of three morphemes: *un-* (not), *believe* (verb root), *-able* (can be done).

Languages handle morphemes very differently (Typology):

1. **Isolating Languages** (e.g., Chinese, Vietnamese): Words mostly consist of single morphemes. Plurality or tense is handled by adding entirely separate words.
2. **Fusional Languages** (e.g., Spanish, Russian): Complex suffixes blend multiple meanings. A single suffix on a Spanish verb conveys person, number, tense, and mood simultaneously.
3. **Agglutinative Languages** (e.g., Turkish, Finnish, Korean): Morphemes are chained together endlessly like train cars. In Turkish, *uygarlaşamadıklarımızdanmışsınızcasına* is a single valid word meaning "as if you were among those whom we could not civilize."

If your system uses a fixed-word dictionary, an agglutinative language will cause an infinite explosion of Out-Of-Vocabulary (OOV) errors. This is the linguistic justification for the subword tokenization mechanisms (BPE) discussed in Chapter 7. Our tokenizers attempt to statistically reverse-engineer morphology.

6.4 Syntax vs. Semantics

In 1957, Noam Chomsky famously proposed the sentence:

Colorless green ideas sleep furiously.

This sentence is **syntactically valid** (it perfectly obeys the grammar rules of English verbs, adjectives, and adverbs). However, it is **semantically void** (it makes no logical sense).

Historically, early NLP tried to code explicit Context-Free Grammars (CFGs) to parse syntax trees. This failed because natural language is riddled with ambiguities that can only be resolved via semantics and pragmatics.

Classical example: *"I saw the man with the telescope."* Did I use a telescope to see the man, or did the man possess a telescope? Syntax cannot resolve this. A neutral language model solves this by looking at broader context (pragmatics). Neural LLMs do not keep explicit parse trees in memory; they project context into continuous vector space, collapsing syntax and semantics into a unified statistical probability.

6.5 Zipf's Law

Zipf's Law is the most important empirical law in NLP. It states that given a large sample of words, the frequency of any word is inversely proportional to its rank in the frequency table.

Let r be the rank of a word (the most frequent word is $r = 1$, the second is $r = 2$). Its frequency f obeys:

$$f(r) \propto \frac{1}{r^{\alpha}} \tag{6.1}$$

where $\alpha \approx 1$.

Implications: 1. **The Heavy Tail**: A tiny handful of words (like "the", "of", "and") make up a massive percentage of any corpus. 2. **The Long Tail**: The vast majority of words in a vocabulary appear extremely rarely. 3. **Data Sparsity**: No matter how large your training dataset is (even trillions of tokens), you will always encounter words or N-grams during testing that you have *never* seen before.

This mathematical property is why the N-gram models of Chapter 8 ultimately failed and why we rely on the continuous space embeddings of neural networks to guess properties of the long-tail words based on their structural similarities to common words.

6.6 Worked Example: Checking Zipf on a Small Sample

In the Brown Corpus of American English (approx. 1 million words): - "the" (Rank 1) appears 69,971 times. - "of" (Rank 2) appears 36,411 times. (Roughly $\frac{1}{2}$ of Rank 1). - "and" (Rank 3) appears 28,852 times. (Roughly $\frac{1}{3}$ of Rank 1).

If we plot log(Rank) vs. log(Frequency), Zipf's Law predicts a straight line with a slope of -1. All human languages exhibit this exact power-law distribution. Language models must allocate an enormous parameter budget (via embeddings) solely to handle the heavy tail of rare concepts.

6.7 Systems and Implementation Consequences

Because of Zipf's Law, when an LLM computes the softmax denominator over a vocabulary of 50,000, it wastes immense amounts of compute driving the probabilities of 49,000 extremely rare words functionally to zero. Optimization techniques like Adaptive Softmax or Hierarchical Softmax historically exploited Zipf's law by clustering rare words together into a sub-tree to avoid computing them unless necessary. However, on modern GPUs with block matrices, hardware accelerates dense, balanced arrays faster than it runs sparse, imbalanced logic trees. Today, we simply bite the bullet and compute the full dense Softmax, favoring hardware symmetry over linguistic asymmetry.

Neuroscience Analogy: Broca's Area vs. Wernicke's Area

Where the analogy helps. Clinical neuroscience historically divided language processing: Broca's Area (frontal lobe) was associated with syntax and language production (grammar), while Wernicke's Area (temporal lobe) was associated with semantics and comprehension (meaning). This mirrors the linguist's split between Syntax and Semantics.

Where the analogy breaks. Modern imaging shows this brain split is highly oversimplified—syntax and semantics are deeply distributed and intertwined networks. In an LLM, there is zero modular separation whatsoever. A single Transformer block handles grammar, meaning, and logic simultaneously in exactly the same monolithic weight matrices.

What intuition to keep. While linguistic theory cleanly separates grammar from meaning, the geometry of high-dimensional continuous representations natively blurs them together. For a neural net, resolving grammar is just another vector operation, identical to retrieving a fact.

6.8 Common Misconceptions

1. **"LLMs learn the grammar rules of English."** LLMs memorize no explicit syntactic rules. They learn a continuous approximation of the probability distribution of language. If a text is grammatically broken but statistically probable (e.g., internet slang), the model treats it as correct.

2. **"Words have fixed meanings."** A linguist will tell you meaning is contextual (Pragmatics). This is why static embeddings (like Word2Vec) were replaced by contextual embeddings (like Transformers), where the mathematical vector for "bank" dynamically changes depending on the surrounding tokens.

6.9 Exercises

Warmup

1. Label the following errors as Syntactic, Semantic, or Pragmatic: (a) "The angry pizza drove explicitly." (b) "Cat the on sat mat." (c) Answering "Yes" to "Do you have the time?" and standing in silence.

Derivation

1. If an agglutinative language builds words exponentially via combinations of N suffixes, mathematically demonstrate why subword tokenization creates a linear $O(N)$ vocabulary size while whole-word tokenization requires an exponential $O(2^N)$ vocabulary.

Implementation

1. [**Prepares for CS336 A4**] Using Python's 'collections.Counter', write a short script that plots the log-rank vs. log-frequency graph for any given text file to verify Zipf's Law.

Open-Ended

1. If an LLM is trained purely on text, does it construct semantics (meaning) or is it just generating highly complex syntax? (Refer to the "Bender's Octopus" thought experiment in NLP literature).

6.10 Further Reading

- *Speech and Language Processing* (Jurafsky & Martin). The gold standard textbook for classical NLP and computational linguistics.
- *Zipf's Law and the Economy of Words* (Original Paper). A deeper dive into why power laws naturally occur in human communication as a tension between speaker effort and listener effort.

Part II

Language Modeling Foundations

Chapter 7

The Language Modeling Problem

Key Insight

After reading this chapter, you will be able to:

1. State the formal definition of autoregressive language modeling.
2. Explain how the chain rule of probability decomposes sequence modeling into next-token prediction.
3. Differentiate between generation (sampling) and scoring (likelihood calculation).
4. Understand the fundamental bottleneck of language modeling: context dependency versus computational feasibility.

7.1 Big Picture

Everything in natural language processing experienced a paradigm shift when researchers realized a profound, counterintuitive truth: predicting the next word in a sequence is not just a parlor trick. It is a conceptually simple objective that, when scaled up, forces a neural network to learn syntax, semantics, logic, and common sense.

To accurately predict the next word in "The cat sat on the ____", a model only needs to know English grammar and common idioms. But to predict the next word in "If we heat the metal to 2000°C, its tensile strength will ____", the model must understand physics and materials science. By training a single architecture to predict the next token across all of human text, we obtain a general-purpose reasoning engine.

This chapter forms the conceptual bridge from the information theory in Chapter 1 to the neural implementations in Chapters 8 and beyond. We establish the fundamental mathematical problem that all generative language models—from n-grams to GPT-4—solve.

7.2 Formal Setup and Notation

We treat language as a sequence of discrete units called **tokens**. (We will discuss exactly how text is split into tokens in Chapter 7; for now, consider them words or characters).

Let our vocabulary be a finite set $\mathcal{V}$. A sequence of length T is denoted $\mathbf{x} = (x_1, x_2, \ldots, x_T)$, where each $x_t \in \mathcal{V}$. The language modeling problem is to estimate the joint probability distribution of the entire sequence:

$$P(X_1 = x_1, X_2 = x_2, \ldots, X_T = x_T) = P(\mathbf{x}). \tag{7.1}$$

Here, X_t is the random variable representing the token at position t, and x_t is the specific realization of that random variable.

7.3 Autoregressive Decomposition

Directly estimating $P(\mathbf{x})$ is intractable. If our vocabulary has $|\mathcal{V}| = 50,000$ and we consider sequences of length $T = 100$, there are $50,000^{100}$ possible sequences. We cannot simply count frequencies in a training corpus; almost all sequences will occur exactly zero times.

To make the problem tractable, we apply the **chain rule of probability**:

$$P(x_1, x_2, \ldots, x_T) = P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_1, x_2) \cdots P(x_T \mid x_1, \ldots, x_{T-1}). \tag{7.2}$$

Using product notation:

$$P(\mathbf{x}) = \prod_{t=1}^{T} P(x_t \mid x_{<t}), \tag{7.3}$$

where $x_{<t} = (x_1, x_2, \ldots, x_{t-1})$ represents the **context** or **history** up to position t.

Symbol table. $P(\mathbf{x})$ is the joint probability of the sequence. $P(x_t \mid x_{<t})$ is the conditional probability of the next token given all previous tokens.

Interpretation. The chain rule tells us that modeling an entire sequence is mathematically equivalent to modeling a single next token, provided we condition on all previous tokens. This transforms language modeling from a massive joint distribution problem into a sequence of classification problems. A model that predicts one step at a time and feeds its own predictions back as input is called **autoregressive**.

7.4 Scoring versus Generation

An autoregressive language model has two primary uses: scoring existing text and generating new text.

7.4.1 Scoring

We can use a language model to compute the likelihood of a given sequence. Recall from Chapter 1 that we typically work with log-probabilities to avoid arithmetic underflow:

$$\log P(\mathbf{x}) = \sum_{t=1}^{T} \log P(x_t \mid x_{<t}). \tag{7.4}$$

This is the heart of model evaluation (perplexity) and training (cross-entropy loss minimization).

7.4.2 Generation

To generate text, we sample from the model one token at a time:

1. Start with a prompt context $x_{<1}$.
2. Sample $x_1 \sim P(\cdot \mid x_{<1})$.
3. Append x_1 to the context: $x_{<2} = (x_{<1}, x_1)$.
4. Sample $x_2 \sim P(\cdot \mid x_{<2})$.
5. Repeat until a special stop token is generated.

7.5 Worked Example: Computing Sequence Probability

Suppose our vocabulary is $\mathcal{V} = \{\text{the}, \text{cat}, \text{sat}\}$. We want to score the sequence $\mathbf{x} = (\text{the}, \text{cat}, \text{sat})$. Our model provides the following conditional probabilities:

$$\begin{aligned} P(x_1 = \text{the}) &= 0.2 \\ P(x_2 = \text{cat} \mid x_1 = \text{the}) &= 0.4 \\ P(x_3 = \text{sat} \mid x_1 = \text{the}, x_2 = \text{cat}) &= 0.8 \end{aligned}$$

The joint probability of the sequence is:

$$\begin{aligned} P(\mathbf{x}) &= P(\text{the}) \times P(\text{cat} \mid \text{the}) \times P(\text{sat} \mid \text{the}, \text{cat}) \\ &= 0.2 \times 0.4 \times 0.8 = 0.064 \end{aligned}$$

In minus log-likelihood (base e):

$$\begin{aligned} -\log(0.064) &= -(\log(0.2) + \log(0.4) + \log(0.8)) \\ &= -(-1.609 - 0.916 - 0.223) = 2.748 \text{ nats} \end{aligned}$$

7.6 Systems and Implementation Consequences

The autoregressive decomposition Equation (7.3) dictates the computational architecture of all modern generative AI.

During training (scoring): Because we are scoring a known sequence $x_1, \ldots, x_T$, the context $x_{<t}$ is fixed for all t. This means we can compute the probabilities for all T positions in parallel. This parallelism is the defining advantage of the Transformer architecture over older recurrent networks.

During inference (generation): Because we must wait to sample x_t before we can append it to the context and predict x_{t+1}, generation is inherently sequential. This makes decoding highly latency-bound and memory bandwidth-bound. We cannot parallelize across time steps during generation. This dichotomy—parallel training, serial inference—drives the engineering constraints discussed in Chapters 14–18.

Neuroscience Analogy: Predictive Coding and Language Generation

Where the analogy helps. The human brain has been theorized to function as a "prediction machine," constantly anticipating upcoming sensory input. Similarly, as you read this sentence, your brain is likely anticipating the next word before your eyes reach it. An autoregressive language model formalizes this exact hypothesis into a mathematical framework.

Where the analogy breaks. Human language generation is hierarchical and goal-directed. A speaker starts with an abstract communicative intent, forms a syntactic chunk, and then articulates words. An autoregressive LLM, by contrast, operates purely left-to-right at the token level, with no explicit internal mechanism separating high-level intent from low-level token choice.

What intuition to keep. Think of the language modeling objective as pure prediction. The model learns everything it knows about grammar and reasoning simply as a byproduct of trying to reduce surprise about the future.

7.7 Common Misconceptions

1. **"Language models understand text, then generate it."** Language models are function approximators mapping contexts to probability distributions. Any semantic "understanding" is entirely implicit in the structure of the conditional distribution $P(x_t \mid x_{<t})$.

2. **"The model predicts the most likely entire sequence."** Usually, models perform greedy decoding (picking the highest probability token at each step) or sampling. Finding the global maximum probability sequence over the entire sequence length is intractable.

3. **"Generative modeling algorithms are specific to text."** The autoregressive formulation applies to any sequence. The exact same objective is used for image generation (treating images as sequences of pixels) and audio generation (as sequences of audio waveforms).

7.8 Exercises

Warmup

1. Suppose a model assigns a probability of 0.1 to the true next token at every position in a sequence of length 5. What is the overall joint probability of the sequence? What is the log-probability?

Derivation

1. Show that if x_t is completely independent of the context $x_{<t}$, the autoregressive model reduces to a unigram model where $P(\mathbf{x}) = \prod_{t=1}^{T} P(x_t)$. What is the context size required to capture all dependencies in natural language?

Implementation

1. [**Prepares for CS336 A1**] Implement a basic loop in Python that demonstrates autoregressive generation given a dummy prediction function `def predict_next_token(context): return token`.

Open-Ended

1. Discuss why bidirectional models (like BERT, which predict missing words given both left and right context) are popular for classification tasks but struggle mathematically with the task of text generation. How does the chain rule map to bidirectionality?

7.9 Further Reading

- "A Neural Probabilistic Language Model" [2]: A Neural Probabilistic Language Model. The seminal paper framing language modeling as a neural task based on the Markov assumption.
- OpenAI [3]: Language Models are Unsupervised Multitask Learners. This OpenAI paper demonstrated that autoregressive next-token prediction, scaled massively, naturally gives rise to zero-shot task performance.

Chapter 8

Tokenization

Key Insight

After reading this chapter, you will be able to:

1. Explain why language models operate on discrete tokens rather than raw text.
2. Describe the trade-offs between word, character, and subword tokenization.
3. Implement the Byte-Pair Encoding (BPE) algorithm from scratch.
4. Understand the Unigram tokenization algorithm and its probabilistic formulation.
5. Discuss the motivation and architecture of byte-level and token-free models.

8.1 Big Picture

Before a neural network can process text, the text must be converted into numbers. This process is called **tokenization**. It sits at the very edge of the model, transforming a continuous stream of characters or bytes into a sequence of discrete integers.

Historically, NLP systems used words as the fundamental unit of text. However, natural language is fraught with complications: morphology ("run", "running", "ran"), agglutination (languages like German or Turkish that glue words together), and typos. If we tokenize by words, our vocabulary grows uncontrollably, leading to the "out-of-vocabulary" (OOV) problem. If we tokenize by characters, the vocabulary is small (e.g., 256 for ASCII), but sequences become extremely long, which computationally burdens the Transformer architecture.

The modern solution is **subword tokenization**. Algorithms like Byte-Pair Encoding (BPE) strike a data-driven balance: common words remain single tokens, while rare words are broken down into smaller, frequent chunks (or individual characters). Nearly every major language model today (GPT-4, LLaMA, DeepSeek) uses a variant of subword tokenization.

8.2 Formal Setup and Notation

Let $\mathcal{C}$ be our base character set (e.g., all 256 possible bytes, or all Unicode characters). A **corpus** is a string sequence over $\mathcal{C}$.

A **tokenizer** consists of two functions:

1. An **encoder**: $E : \mathcal{C}^* \to \mathcal{V}^*$, mapping a string of characters to a sequence of token IDs from a fixed integer vocabulary $\mathcal{V} = \{1, 2, \ldots, V\}$.

2. A **decoder**: $D : \mathcal{V}^* \to \mathcal{C}^*$, mapping a sequence of token IDs back to a string.

A proper tokenizer must be lossless: for any valid text s, $D(E(s)) = s$.

Vocabulary size. The size of the vocabulary V is a crucial hyperparameter. Typical values range from $32,000$ (LLaMA-2) to $128,000$ (GPT-4). A larger vocabulary means shorter sequences (good for compute) but requires a larger embedding matrix (bad for memory).

8.3 Byte-Pair Encoding (BPE)

BPE was originally a data compression algorithm introduced by "Sennrich" [4] to the NLP world. It is a greedy, bottom-up algorithm that builds a vocabulary by iteratively merging the most frequent pairs of units.

8.3.1 The BPE Training Algorithm

To "train" a BPE tokenizer on a corpus, we perform the following steps:

1. Initialize the vocabulary with the base vocabulary $\mathcal{C}$ (usually all 256 ASCII/UTF-8 bytes).

2. Represent the training corpus as a sequence of these base tokens.

3. Count the frequency of all adjacent token pairs.

4. Find the most frequent pair (t_1, t_2).

5. Create a new token $t_{\text{new}} = t_1 \circ t_2$ by concatenating them.

6. Add t_{new} to the vocabulary.

7. Replace all non-overlapping occurrences of the pair (t_1, t_2) in the corpus with t_{new}.

8. Repeat steps 3–7 until the vocabulary reaches the desired size V.

8.3.2 Encoding with BPE

Once trained, the sequence of merges defines the encoding process. Given a new string, we first split it into base tokens (bytes). We then apply the merge operations in the exact same order they were learned during training.

Common Pitfall

Order matters! If "e" + "r" → "er" was learned before "t" + "h" → "th", you must apply the "er" merge to your new text before looking for "th".

8.4 The Unigram Language Model Tokenizer

While BPE is bottom-up and greedy, the **Unigram** tokenizer operates top-down and probabilistically.

1. Initialize a massive vocabulary of potential subwords (e.g., all substrings up to length L in the corpus).
2. Train a unigram language model over these subwords. Let $p(x_i)$ be the probability of subword x_i. The probability of a tokenization $\mathbf{x} = (x_1, \ldots, x_T)$ is assumed to be $\prod p(x_i)$.
3. Find the optimal tokenization for the corpus (using the Viterbi algorithm) to maximize likelihood under the current probabilities.
4. Compute the "loss" for each subword: how much does the overall log-likelihood drop if we remove this subword from the vocabulary?
5. Discard the bottom 20% of subwords (those with the smallest loss).
6. Repeat until the vocabulary shrinks to the desired size V.

Unigram tokenization has the advantage of supporting **subword regularization**: because it is probabilistically grounded, we can sample different valid tokenizations for the same word during training, which makes the model more robust to misspellings.

8.5 Worked Example: BPE Training

Suppose our training corpus is the string: `l o w e s t l o w e r` (We insert spaces merely to show the current token boundaries).

Initialization: Base vocabulary: $\{\mathrm{l}, \mathrm{o}, \mathrm{w}, \mathrm{e}, \mathrm{s}, \mathrm{t}, \mathrm{r}\}$

Iteration 1: Frequencies of adjacent pairs:

- (l, o): 2

- (o, w): 2
- (w, e): 2
- (e, s): 1
- (s, t): 1
- (t, l): 1
- (e, r): 1

The pair (l, o) is tied for the most frequent. We merge it. New token: `lo`. Corpus becomes: `lo w e s t lo w e r`

Iteration 2: Frequencies of pairs:

- (`lo`, w): 2
- (w, e): 2
- (e, s): 1 ...

Merge (`lo`, w). New token: `low`. Corpus becomes: `low e s t low e r`

Iteration 3: Merge (w, e)? No, "w" is now part of "low". The pair (`low`, e) occurs twice. Merge it. New token: `lowe`. Corpus becomes: `lowe s t lowe r`

After 3 merges, our vocabulary is $\{\mathrm{l}, \mathrm{o}, \mathrm{w}, \mathrm{e}, \mathrm{s}, \mathrm{t}, \mathrm{r}, \mathrm{lo}, \mathrm{low}, \mathrm{lowe}\}$. The word "lowest" is tokenized as [`lowe`, s, t], and "lower" is [`lowe`, r]. The tokenizer learned the root morphological structure purely statistically!

8.6 Systems and Implementation Consequences

Tokenization is notoriously complex in engineering systems:

- **Pre-tokenization:** Before BPE, we typically use regex to split text into words and punctuation. If we don't, BPE might create tokens that span across word boundaries (e.g., merging the end of one word and the start of the next), which damages syntactic generalization.
- **Byte-Fallbacks:** To ensure the tokenizer can encode *any* string without "[UNK]" (unknown) tokens, modern tokenizers (like LLaMA's) operate on UTF-8 bytes rather than Unicode characters.
- **Embedding Matrix Size:** The dimension of the first and last weight matrices in the model is exactly $V \times d$. For $V = 100,000$ and $d = 4096$, the embedding matrix takes 1.6 GB of memory in fp32. This is often a significant portion of a smaller model's parameter count.

Neuroscience Analogy: Phonemes and Visual Features vs. Tokens

Where the analogy helps. Just as humans decompose spoken language into a finite set of phonemes (sound units) or written words into visual strokes/features, an LLM decomposes text into a finite set of subword tokens. In both cases, the complex continuous signal of language is chunked into discrete components that the processing system can handle.

Where the analogy breaks. Human chunking is profoundly influenced by semantics and morphology; we parse "unbelievable" into prefix, root, and suffix because of their meanings. BPE is purely statistical and frequency-driven. If a typo appears often enough in the training data, BPE will happily dedicate a dedicated token to it.

What intuition to keep. Tokens are the fundamental "atoms" of meaning for the model. However, these atoms are derived from compression algorithms, not linguistic theory. The model's "understanding" of a word is heavily dependent on how the tokenizer happened to slice it.

2026 Update: Byte-Level and Token-Free Models

While BPE is dominant, the field is actively researching ways to eliminate tokenizers entirely. Operating directly on raw bytes removes the complexity of the tokenizer pipeline and prevents issues where BPE behaves poorly on heavily numeric data (arithmetic), code spacing, or languages underrepresented in the tokenizer's training data.

Systems like **Megabyte** [**megabyte_2023**] and **BLT** [**blt_2024**] propose hierarchical architectures: a small recurrent or local module processes raw bytes and clusters them into "patch" representations, which are then processed by a global Transformer. This provides the flexibility of bytes without the massive sequence-length scaling penalty in the attention mechanism.

8.7 Common Misconceptions

1. **"One token equals one word."** On average in English, 1 token is about 0.75 words, or 4 characters. But common words ("the") are 1 token, while rare/complex words ("neuroscience") might be 2 or 3 tokens.

2. **"The model sees the characters inside a token."** It does not. If "cat" is tokenized as ID #2503, the model simply receives a vector mapping to #2503. It has no way of knowing that "c", "a", and "t" are the constituent letters unless it deduces it statistically from other tokens containing those letters. This is why LLMs struggle with character-level tasks like "how many r's in strawberry".

3. **"Tokenization is learned alongside the language model."** Tokenization is completely disjoint. The tokenizer is trained once on a corpus to build a fixed mapping dictionary.

The neural network is trained later using that fixed dictionary.

8.8 Exercises

Warmup

1. If your text corpus has $1,000,000$ characters and your vocabulary size is 1000, what is the absolute minimum and maximum number of tokens it could take to encode the corpus?

2. Why do LLMs typically learn a completely separate token for a capitalized word (e.g., "Apple") vs its lowercase version ("apple"), rather than sending a "capitalization" token?

Derivation

1. Let the vocabulary size V grow arbitrarily large. Trace the impact of this on (a) sequence length, (b) embedding memory usage, and (c) the probability of an OOV (out-of-vocabulary) word. What prevents us from setting $V = 1,000,000$?

Implementation

1. [**Prepares for CS336 A1**] Write a Python script to train a basic BPE tokenizer. Input: a list of strings (using characters as initial tokens). Output: a sequence of merge rules. Show the state of your vocabulary after 10 merges.

Open-Ended

1. Discuss how BPE tokenization impacts multi-lingual language models. If the training data corpus is 90% English and 10% Chinese, what will the resulting BPE vocabulary look like, and how will this affect the efficiency of inferencing Chinese text?

8.9 Further Reading

- "Sennrich" [4]: Neural Machine Translation of Rare Words with Subword Units. The paper that introduced BPE to NLP.

- **blt_2024**: BLT: Byte Latent Transformer. A cutting-edge architecture exploring how to replace subword tokenization entirely with byte-level latent processing.

- **byt5_2021**: ByT5: Towards a token-free future with pre-trained byte-to-byte models.

Chapter 9

From N-Grams to Neural Language Models

Key Insight

After reading this chapter, you will be able to:

1. Construct an n-gram language model and explain its fundamental limitations (the curse of dimensionality).
2. Describe how continuous vector embeddings solve the data sparsity problem.
3. Formulate a simple Neural Probabilistic Language Model (NPLM).
4. Understand why recurrent neural networks (RNNs) failed to scale for long context lengths.

9.1 Big Picture

Before Transformers dominated NLP, the field progressed through two distinct epochs: the n-gram era and the early neural era. This chapter covers both, not just for historical context, but because understanding *why* these older methods failed provides the precise motivation for the mathematical structure of the Transformer.

The n-gram method represents language as exact counts of discrete sequences. It is simple and fast, but it suffers from the "curse of dimensionality"—it cannot generalize to sequences it has never seen before. The first neural language models fixed this by embedding discrete tokens into a continuous vector space, allowing them to generalize based on geometric similarity. However, their architectures (feed-forward or recurrent) struggled to efficiently route information across long distances. This sets the stage appropriately for Chapter 9, where we introduce the attention mechanism.

9.2 Formal Setup and Notation

Recall the language modeling problem from Chapter 6: we want to estimate $P(x_t \mid x_{<t})$. Let $C(w_1, \ldots, w_k)$ denote the number of times the exact sequence of tokens $(w_1, \ldots, w_k)$ appears in our training corpus.

We map each token $x_i \in \mathcal{V}$ to an integer index $v \in \{1, \ldots, V\}$. A **one-hot vector** $\mathbf{e}_v \in \{0, 1\}^V$ is a vector where the v-th element is 1 and all other elements are 0.

An **embedding matrix** $\mathbf{W}_E \in \mathbb{R}^{V \times d}$ maps tokens to continuous vectors in d-dimensional space. The embedding for token v is exactly the v-th row of $\mathbf{W}_E$, which can be mathematically written as $\mathbf{x} = \mathbf{e}_v^\mathsf{T} \mathbf{W}_E \in \mathbb{R}^d$.

9.3 The N-Gram Language Model

An n-gram model makes a simplifying Markov assumption: the probability of the next token depends *only* on the preceding $n-1$ tokens.

$$P(x_t \mid x_1, \ldots, x_{t-1}) \approx P(x_t \mid x_{t-n+1}, \ldots, x_{t-1}). \tag{9.1}$$

To estimate this probability, we use maximum likelihood estimation based on counts:

$$P_{\text{MLE}}(x_t = v \mid x_{t-n+1}, \ldots, x_{t-1}) = \frac{C(x_{t-n+1}, \ldots, x_{t-1}, v)}{C(x_{t-n+1}, \ldots, x_{t-1})}. \tag{9.2}$$

9.3.1 The Curse of Dimensionality

Suppose we build a 5-gram model over a vocabulary of $V = 50,000$. There are $V^4 = 6.25 \times 10^{18}$ possible histories of length 4. Even a massive corpus like Common Crawl (trillions of tokens) will not contain the vast majority of these histories.

If we encounter a history h that never appeared in the training set, $C(h) = 0$, and the probability is undefined. If we encounter a history that appeared, but a valid next word never followed it in the training set, $P_{\text{MLE}} = 0$, resulting in infinite cross-entropy (-log $0 = \infty$).

To fix this, n-gram models use **smoothing** (e.g., Kneser-Ney), which falls back to lower-order models (4-grams, then 3-grams) when data is sparse. But smoothing does not solve the fundamental flaw: n-gram models treat tokens as atomic symbols. They do not know that "cat" and "dog" are similar. If a model learns that "A dog is sleeping in the _____" predicts "bed", it cannot generalize this to "A cat is sleeping in the _____" unless that exact 6-gram also appeared.

9.4 Worked Example: Trigram Calculation

Suppose our corpus is exactly the following three sentences:

- `<s> i love deep learning </s>`
- `<s> i love machine learning </s>`
- `<s> cats love deep sleeping </s>`

We want to build a trigram ($n = 3$) model and compute the probability of "`learning`" given the history "`love deep`".

1. Count the history: $C(\texttt{love}, \texttt{deep}) = 2$. 2. Count the full trigram: $C(\texttt{love}, \texttt{deep}, \texttt{learning}) = 1$. 3. Compute MLE:

$$P_{\text{MLE}}(\texttt{learning} \mid \texttt{love}, \texttt{deep}) = \frac{1}{2} = 0.5. \tag{9.3}$$

Now, compute the probability of "`learning`" given "`love machine`": 1. $C(\texttt{love}, \texttt{machine}) = 1$. 2. $C(\texttt{love}, \texttt{machine}, \texttt{learning}) = 1$. 3. $P_{\text{MLE}} = \frac{1}{1} = 1.0$.

9.5 The Neural Probabilistic Language Model (NPLM)

In 2003, Bengio et al. proposed a solution to the curse of dimensionality: represent each token as a continuous vector. If the vectors for "cat" and "dog" are close in the d-dimensional space, the neural network's smooth prediction function will automatically assign similar probabilities to contexts involving them.

A classic window-based neural language model computes $P(x_t \mid x_{t-n+1}, \ldots, x_{t-1})$ via three steps:

1. **Embed**: Look up the vector for each of the $n - 1$ context tokens.
2. **Concatenate**: Flatten these vectors into a single vector $\mathbf{h}_{\text{in}} \in \mathbb{R}^{(n-1)d}$.
3. **Project**: Pass $\mathbf{h}_{\text{in}}$ through a multi-layer perceptron (MLP) to get a hidden state, then project it to the vocabulary size to get **logits** $\mathbf{z} \in \mathbb{R}^V$.
4. **Softmax**: Convert logits to probabilities.

$$P(x_t = v \mid \text{context}) = \text{softmax}(\mathbf{z})_v = \frac{\exp(z_v)}{\sum_{k=1}^{V} \exp(z_k)}. \tag{9.4}$$

9.6 The Recurrent Era and Its Failure Modes

Window-based feed-forward neural networks still suffer from a fixed context window n. To allow infinite context, researchers turned to Recurrent Neural Networks (RNNs) and their variants (LSTMs, GRUs). In an RNN, the hidden state at time t depends on the input at time t and the hidden state at time $t - 1$:

$$\mathbf{h}_t = f(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}). \tag{9.5}$$

Why did RNNs lose to Transformers?

1. **Vanishing Gradients**: Gradients must flow backwards through time (BPTT). Over hundreds of steps, the repeated multiplication by the recurrent weight matrix $\mathbf{W}_h$ causes gradients to vanish or explode, making it impossible to learn long-term dependencies.

2. **State Bottleneck**: The entire history of the sequence must be compressed into a single, fixed-size vector $\mathbf{h}_t$. If the sequence is 10,000 tokens long, $\mathbf{h}_t$ struggles to remember the first 100 tokens.

3. **Lack of Parallelism**: To compute $\mathbf{h}_t$, you strictly must compute $\mathbf{h}_{t-1}$ first. This serial dependency makes RNNs fundamentally hostile to GPU hardware, which excels at doing thousands of operations simultaneously. During training, we know the whole sequence, but an RNN still forces us to process it sequentially.

These three flaws—vanishing gradients, state bottlenecks, and serial execution—are exactly the problems the Transformer's attention mechanism solves.

9.7 Systems and Implementation Consequences

The transition from n-grams to neural models shifted NLP from memory-bound table lookups to compute-bound dense linear algebra.

- **Embedding tables** are sparse lookups. We do not actually multiply $\mathbf{e}_v^\top \mathbf{W}_E$ (which is mostly zeros), we simply index into the v-th row in PyTorch using `nn.Embedding`.

- **The Softmax Bottleneck**: The final projection matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$ is massive. If $d = 4096$ and $V = 100,000$, computing the logits takes a huge number of FLOPs. The softmax operation itself requires calculating the sum of exponentials across the entire vocabulary V for every batch item and token position. Early NPLMs used approximations like hierarchical softmax or negative sampling to avoid computing the full denominator, but modern hardware is fast enough that we simply compute the full softmax.

Neuroscience Analogy: Population Coding and Embeddings

Where the analogy helps. How does the brain represent the concept of "apple"? It is not a single "apple neuron" firing (a one-hot encoding). Instead, it is a pattern of activation across millions of neurons (a distributed representation). If you see a pear, the neural pattern will be highly overlapping with the apple pattern. This is precisely how continuous vector embeddings work in neural networks: similar concepts map to vectors with high cosine similarity.

Where the analogy breaks. Neural networks use dense float vectors where every dimension participates in every concept. The brain uses sparse coding—only a small, specialized subset of neurons fires for any given stimulus, which is vastly more energy efficient.

What intuition to keep. Moving from distinct n-gram symbols to continuous vectors enables generalization. A model learns about a new word simply by mapping its vector near the vectors of similar words it already knows.

9.8 Common Misconceptions

1. **"Embeddings are fixed mappings from words to meaning."** While Word2Vec embeddings were static, modern language models compute contextual embeddings. The vector for "bank" in "river bank" is entirely different from "bank" in "bank account" in the deeper layers of a modern model, though the initial lookup vector is the same.

2. **"RNNs failed because they couldn't model language."** RNNs and LSTMs were highly successful language models. They failed because they could not scale on GPU hardware. The Transformer won not because it has a better linguistic inductive bias, but because it is massively parallelizable, allowing us to train on exponentially more data.

9.9 Exercises

Warmup

1. In a 4-gram model over a vocabulary of 10,000, how many possible 4-grams exist?

2. If an embedding matrix is $100,000 \times 4096$ using 16-bit floats (2 bytes per number), how much memory does it consume?

Derivation

1. Prove mathematically why repeatedly multiplying a vector by a matrix $\mathbf{W}$ (as in a simplified RNN $\mathbf{h}_t = \mathbf{W}\mathbf{h}_{t-1}$) whose largest eigenvalue is less than 1 leads to exponentially decaying

gradients (the vanishing gradient problem).

Implementation

1. [**Prepares for CS336 A1**] Write a PyTorch class `SimpleNPLM` that inherits from `nn.Module`. It should take a sequence of $(n-1)$ integer tokens, map them through an `nn.Embedding`, concatenate them, pass them through a linear layer and a ReLU, and finally output logits over the vocabulary.

Open-Ended

1. If RNNs suffered from a state bottleneck (compressing all history into one vector), how do you think modern architectures like the Transformer solve this? (Preview your intuition before Chapter 9).

9.10 Further Reading

- "A Neural Probabilistic Language Model" [2]: A Neural Probabilistic Language Model. Introduces word embeddings and the neural approach to curing the curse of dimensionality.
- **mikolov2013**: Efficient Estimation of Word Representations in Vector Space (Word2Vec). A milestone paper explaining the geometric properties of word embeddings.

Part III

Transformer Internals

Chapter 10

Attention Mechanisms

Key Insight

After reading this chapter, you will be able to:

1. Understand the mathematical motivation for the attention mechanism as a differentiable dictionary lookup.
2. Derive the scaled dot-product attention equation and explain the need for the scaling factor $\frac{1}{\sqrt{d_k}}$.
3. Construct and apply a causal mask for autoregressive sequence modeling.
4. Explain how Multi-Head Attention allows a model to represent multiple, independent relationships between tokens.

10.1 Big Picture

As we saw in Chapter 8, older architectures like RNNs compress the entire history of a sequence into a single, fixed-size vector. This creates an unmanageable information bottleneck as sequences grow longer.

The **Attention Mechanism** solves this by discarding compression entirely. Instead of trying to remember the past, an attention layer looks at the *exact* past tokens at every step. It acts like a powerful, differentiable search engine: each token generates a "query" about what information it needs, scans all previous tokens' "keys" to find matches, and then extracts their "values".

This mechanism is the core of the Transformer architecture. Every other component in an LLM exists solely to feed, normalize, or process the outputs generated by attention.

10.2 Formal Setup and Notation

Let $\mathbf{X} \in \mathbb{R}^{T \times d_m}$ be a sequence of T tokens, where d_m is the model dimension. The attention mechanism uses three projection matrices to map the input $\mathbf{X}$ into three distinct spaces:

- **Queries** ($\mathbf{Q} = \mathbf{X}\mathbf{W}_Q \in \mathbb{R}^{T \times d_k}$): What each token is looking for.
- **Keys** ($\mathbf{K} = \mathbf{X}\mathbf{W}_K \in \mathbb{R}^{T \times d_k}$): What each token contains.
- **Values** ($\mathbf{V} = \mathbf{X}\mathbf{W}_V \in \mathbb{R}^{T \times d_v}$): The actual information to be extracted.

Typically, $d_k = d_v$. We will assume d_k is the attention dimension.

10.3 Scaled Dot-Product Attention

The core operation computes the interaction between every query and every key using a dot product, applies a softmax to convert these unbounded scores into probabilities, and uses them to compute a weighted sum of the values.

The canonical equation is:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\mathsf{T}}{\sqrt{d_k}}\right)\mathbf{V} \tag{10.1}$$

10.3.1 The Pre-Softmax Scaling Factor

Why divide by $\sqrt{d_k}$? Assume the elements of $\mathbf{Q}$ and $\mathbf{K}$ are independent random variables with zero mean and variance 1. The dot product of a query vector $\mathbf{q} \in \mathbb{R}^{d_k}$ and a key vector $\mathbf{k} \in \mathbb{R}^{d_k}$ is $\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$. Because this is a sum of d_k independent terms, its variance is d_k.

If $d_k = 64$, the variance of the dot product is 64, meaning the pre-softmax scores will have huge magnitudes. The softmax function $\frac{\exp(x_i)}{\sum \exp(x_j)}$ will be pushed deep into regions where the gradient ≈ 0, stalling training. Dividing the dot product by $\sqrt{d_k}$ forces the variance back to 1.

10.4 Causal Masking

Equation Equation (10.1) computes the interaction between *all* tokens. However, in an autoregressive language model (Chapter 6), predicting token x_t must only depend on $x_{<t}$. If token t attends to token $t+1$, the model is "cheating" by looking into the future during training.

We prevent this by adding a **mask matrix** $\mathbf{M} \in \mathbb{R}^{T \times T}$ before the softmax:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases} \tag{10.2}$$

Thus, the causal attention equation becomes:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\mathsf{T}}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V} \tag{10.3}$$

Because $\exp(-\infty) = 0$, the attention weights for future tokens are strictly zero.

10.5 Multi-Head Attention (MHA)

A single attention mechanism computing a single weighted sum of values is restrictive. What if a "verb" token needs to attend to its "subject" token to check plural agreement, but simultaneously needs to attend to an "adverb" token to check semantics?

We solve this by running h independent attention mechanisms (heads) in parallel.

1. Project $\mathbf{X}$ into h different sets of queries, keys, and values.

2. Compute Equation Equation (10.3) for each head independently.

3. Concatenate the h output matrices.

4. Linearly project the concatenated matrix back to the model dimension d_m using a matrix $\mathbf{W}_O$.

To keep parameters and compute constant regardless of h, we split the dimensions: $d_k = d_v = \frac{d_m}{h}$. If $d_m = 4096$ and $h = 32$, each head operates in a 128-dimensional subspace.

10.6 Worked Example: Attention Matrix Shapes

Given: batch size $B = 4$, sequence length $T = 10$, model dimension $d_m = 512$, and heads $h = 8$. What are the shapes of all intermediate tensors when executed in PyTorch?

1. The input $\mathbf{X}$ has shape $(B, T, d_m) = (4, 10, 512)$. 2. We project to $\mathbf{Q}, \mathbf{K}, \mathbf{V}$. The query projection matrix $\mathbf{W}_Q$ has shape $(512, 512)$. (Because $h \times d_k = 8 \times 64 = 512$). 3. The resulting $\mathbf{Q}$ has shape $(4, 10, 512)$. We reshape and transpose it to isolate the heads: $(B, h, T, d_k) = (4, 8, 10, 64)$. 4. We compute $\mathbf{Q}\mathbf{K}^\mathsf{T}$. We want the dot product over the d_k dimension. Shape: $(4, 8, 10, 64) \times (4, 8, 64, 10) \to (4, 8, 10, 10)$. This is the 10×10 attention score matrix for each head and batch. 5. Apply causal mask (shape 10×10 broadcasted) and softmax. 6. Multiply by $\mathbf{V}$: $(4, 8, 10, 10) \times (4, 8, 10, 64) \to (4, 8, 10, 64)$. 7. Reshape (concatenate heads): $(4, 10, 512)$. 8. Final projection by $\mathbf{W}_O$ (shape 512×512) yields $(4, 10, 512)$.

Notice that the input and output shapes of the entire Multi-Head Attention block are identical.

10.7 Systems and Implementation Consequences

The attention matrix $\mathbf{S} = \mathbf{Q}\mathbf{K}^\mathsf{T}$ has shape $T \times T$. This establishes the fundamental scaling law of the vanilla Transformer: memory and compute scale **quadratically** $O(T^2)$ with the sequence length.

For $T = 1000$, 1000×1000 requires minimal memory. For a document where $T = 100,000$, the $T \times T$ matrix alone would swallow gigabytes of VRAM per batch per layer. As we will see in Chapter 16, avoiding the materialization of this $T \times T$ matrix using IO-aware kernels (FlashAttention) is the most critical systems optimization in modern ML.

Neuroscience Analogy: The Cocktail Party Effect and Selective Attention

Where the analogy helps. At a loud party, your brain can dynamically suppress background noise and isolate one specific person's voice (the "target"). The Transformer's attention mechanism behaves similarly: given a word (the query), the softmax distribution suppresses irrelevant context words (zeroing out their weights) and attends sharply to the geometrically relevant words (the keys), routing only their information forward.

Where the analogy breaks. Human selective attention happens in real-time sequential processing and heavily involves physical sensory gating via the thalamus. Neural network attention operates instantly across all past tokens simultaneously via dense linear algebra. Furthermore, human attention is tightly bottlenecked (we can generally only focus on one conversational stream), whereas Multi-Head Attention computes dozens of independent attentional streams in parallel without interference.

What intuition to keep. Attention is a routing mechanism. Instead of pushing all text through rigid, fixed pathways, the data itself determines what connections matter for each specific word.

10.8 Common Misconceptions

1. **"Attention is all you need."** The title of the paper is catchy, but false. Attention merely routes information. If you remove the heavy Feed-Forward Networks (MLPs) from the Transformer layer, the model entirely loses its ability to store facts and perform complex non-linear reasoning.

2. **"Attention weights show exactly how the model thinks."** Visualizing the softmax weights 10×10 matrix gives a nice graphic associating "it" with "the animal", but it is misleading. Because of value vectors and subsequent layers mixing the data, high attention weight does not definitively prove causal importance.

10.9 Exercises

Warmup

1. If $\mathbf{Q} \in \mathbb{R}^{100\times 64}$, how many operations (multiply-accumulates) are required to compute $\mathbf{QK}^\mathsf{T}$?

2. In a causal mask, why do we use $-\infty$ instead of 0?

Derivation

1. Prove mathematically why dividing by $\sqrt{d_k}$ ensures the variance of the dot product $\mathbf{q} \cdot \mathbf{k}$ is 1, given $q_i \sim \mathcal{N}(0, 1)$ and $k_i \sim \mathcal{N}(0, 1)$.

Implementation

1. [**Prepares for CS336 A1**] Implement a PyTorch function `causal_attention(q, k, v)` using raw tensor operations and `torch.einsum`. Verify that the attention weights in the upper triangle are exactly zero after the softmax.

Open-Ended

1. Since causal attention restricts a token to only look backwards, does this mean token x_{10} can never be influenced by token x_{11} during backpropagation training? Think carefully about the flow of gradients.

10.10 Further Reading

- **vaswani2017**: Attention Is All You Need. The foundational paper that defined scaled dot-product attention and MHA.

- **bahdanau2014**: Neural Machine Translation by Jointly Learning to Align and Translate. The pre-Transformer origin of the attention mechanism in RNNs.

Chapter 11

Transformer Blocks from Scratch

Key Insight

After reading this chapter, you will be able to:

1. Construct a full Transformer block, integrating Multi-Head Attention, MLPs, LayerNorm, and Residual Connections.
2. Explain the necessity of the SwiGLU / Feed-Forward layer for storing world knowledge.
3. Distinguish between Pre-Norm and Post-Norm architectures and explain why Pre-Norm is used in all modern LLMs.
4. Understand the mathematical function of Residual Connections in mitigating the vanishing gradient problem.

11.1 Big Picture

Chapter 9 gave us the engine (Attention) that allows tokens to talk to each other. But an engine is useless without a chassis, transmission, and fuel.

A **Transformer Block** is the self-contained, repeated unit that comprises a language model. It takes an input sequence $\mathbf{X}$, allows the tokens to communicate via Attention, updates their individual internal representations via a Feed-Forward network, and stabilizes the signal via normalization and residual pathways, outputting a new sequence $\mathbf{X}'$ of the exact same shape.

By stacking N of these blocks identically (e.g., $N = 12$ for GPT-2, $N = 80$ for LLaMA-2), the network gradually refines its representation of the text from shallow lexical features (in the first layers) to deep semantic concepts (in the final layers) until the final representation is rich enough to predict the next token.

11.2 Formal Setup and Notation

Let $\mathbf{x} \in \mathbb{R}^{d_m}$ be the representation of a single token at some intermediate layer. A Transformer block applies a sequence of non-linear transformations to $\mathbf{x}$.

- **Attention**: $\text{MHA}(\mathbf{X})$. Mixes information across the sequence length T.
- **Feed-Forward**: $\text{FFN}(\mathbf{x})$. Mixes information across the model dimension d_m, independently for each token.
- **LayerNorm**: $\text{LN}(\mathbf{x})$. Standardizes the vector to mean 0, variance 1.

11.3 The Residual Connection

Deep neural networks suffer from optimization collapse: as the network gets deeper, gradients vanish. He et al. solved this by introducing the **Residual Connection** (or Skip Connection).

Instead of a layer computing the *new* representation directly ($f(\mathbf{x}) = \mathbf{x}_{\text{new}}$), the layer only computes the *change* or *residual* ($\Delta\mathbf{x}$). The operation becomes:

$$\mathbf{x}_{\text{new}} = \mathbf{x} + \text{Sublayer}(\mathbf{x}) \tag{11.1}$$

Mathematically, when you take the derivative of the loss with respect to $\mathbf{x}$, you get:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{new}}} \left(\mathbf{I} + \frac{\partial \text{Sublayer}(\mathbf{x})}{\partial \mathbf{x}} \right) \tag{11.2}$$

The identity matrix $\mathbf{I}$ guarantees that the gradient can always flow backward cleanly, completely bypassing the complex derivatives of the sublayer if necessary.

11.4 Layer Normalization: Pre-Norm vs Post-Norm

To keep activations from exploding as we sum hundreds of residuals, we apply Layer Normalization.

$$\text{LN}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta} \tag{11.3}$$

(We will cover RMSNorm, a modern optimization, in Chapter 11).

The original 2017 Transformer used **Post-Norm**: $\mathbf{x} = \text{LN}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$

This proved highly unstable to train from scratch for very deep models. The gradients through the massive sequence of LayerNorms degraded. Currently, all major LLMs use **Pre-Norm**: $\mathbf{x} = \mathbf{x} + \text{Sublayer}(\text{LN}(\mathbf{x}))$

In Pre-Norm, the residual pathway $\mathbf{x}_{\text{in}} \to \mathbf{x}_{\text{out}}$ never passes through *any* non-linearities or normalizations. The entire LLM backbone is just one uninterrupted addition highway from layer 1 to layer N.

11.5 The Position-Wise Feed-Forward Network (FFN)

Attention computes *which* tokens should interact. But once token A receives the vector representation from token B, it must update its internal state to reflect that new knowledge. This requires an MLP.

The standard Transformer FFN is a two-layer perceptron applied identically to every token:

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 \tag{11.4}$$

Crucially, the hidden dimension of this FFN is traditionally heavily expanded, usually to $4 \times d_m$. If $d_m = 4096$, the FFN projects the token up to 16384 dimensions, applies a non-linearity, and projects it back down to 4096.

Why the FFN? Attention has no non-linearities (other than the routing softmax). It is just a weighted average of vectors. The FFN provides the non-linear capacity required to memorize facts. Recent interpretability research conceptualizes the FFN as a Key-Value Memory Bank: $\mathbf{W}_1$ acts as a set of keys detecting specific linguistic patterns in $\mathbf{x}$, and $\mathbf{W}_2$ outputs the corresponding associated knowledge values back into the residual stream.

Modern models (like LLaMA) replace the ReLU MLP with a **SwiGLU** variant, which performs a gating mechanism:

$$\text{SwiGLU}(\mathbf{x}) = (\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot (\mathbf{x}\mathbf{W}_V))\mathbf{W}_2 \tag{11.5}$$

We will code this explicitly in Assignment 1.

11.6 Assembling the Final Block

With these components, the standard decoder-only Transformer Block (using Pre-Norm) is exactly two equations:

$$\mathbf{x}_{\text{mid}} = \mathbf{x} + \text{MHA}(\text{LN}(\mathbf{x})) \tag{11.6}$$

$$\mathbf{x}_{\text{out}} = \mathbf{x}_{\text{mid}} + \text{FFN}(\text{LN}(\mathbf{x}_{\text{mid}})) \tag{11.7}$$

No magic. Just an input vector $\mathbf{x}$, two normalizations, an attention sweep and a neural expansion, added back to the input. We replicate this block N times.

11.7 Worked Example: Parameter Counting

Let's calculate the exact number of learnable parameters in a single block where $d_m = 512$, $h = 8$, and the FFN expansion is $4\times$ ($d_{\text{ff}} = 2048$). We assume no bias vectors, conforming to modern LLM standards (like PaLM and LLaMA).

1. **Attention Sublayer**: - $\mathbf{W}_Q \in \mathbb{R}^{512\times512} \implies 262,144$ parameters. - $\mathbf{W}_K \in \mathbb{R}^{512\times512} \implies 262,144$ parameters. - $\mathbf{W}_V \in \mathbb{R}^{512\times512} \implies 262,144$ parameters. - $\mathbf{W}_O \in \mathbb{R}^{512\times512} \implies 262,144$ parameters. - *Subtotal:* $1,048,576$ *parameters.*

2. **FFN Sublayer**: - $\mathbf{W}_1 \in \mathbb{R}^{512\times2048} \implies 1,048,576$ parameters. - $\mathbf{W}_2 \in \mathbb{R}^{2048\times512} \implies 1,048,576$ parameters. - *Subtotal:* $2,097,152$ *parameters.*

3. **LayerNorms** (2 per block): - $\boldsymbol{\gamma}_1, \boldsymbol{\gamma}_2 \in \mathbb{R}^{512} \implies 1,024$ parameters. - *Subtotal:* $1,024$ *parameters.*

Total per block: $3,146,752$ parameters. Notice that the FFN contains $\sim 66\%$ of the parameters in the entire block. The "Attention" mechanism is actually the minority shareholder in an LLM's capacity!

11.8 Systems and Implementation Consequences

Memory limits everything. During training, Autograd must cache the input to *every* matrix multiplication to compute the backward pass. For a batch size of B, sequence length T, and model dimension d, caching $\mathbf{x}_{\text{in}}$ across 80 blocks consumes monumental amounts of VRAM. This leads to **Activation Checkpointing** (or Gradient Checkpointing): instead of saving the activations in memory, we simply delete them during the forward pass, and physically *recompute* them from scratch during the backward pass. We trade TFLOPs of compute to save Gigabytes of VRAM.

Neuroscience Analogy: Cortical Columns and the Residual Stream

Where the analogy helps. The neocortex is physically structured into vertical columns where sensory information flows sequentially up through layers. At every step, local computation creates complex representations, and the result is passed upward. The Transformer block is an exact computational analog to this layered processing. The residual connection acts like the heavy vertical axonal bundles that ensure raw sensory signals don't degrade entirely before reaching higher cognitive centers.

Where the analogy breaks. The brain possesses massive top-down feedback loops (higher layers predicting lower layers). A Transformer is strictly feedforward during a single timestep.

What intuition to keep. The residual stream ($\mathbf{x}$) is a global "bulletin board" that persists from the input of the model all the way to the output. Every block (column) reads from this bulletin board (via LN), computes some specialized insight (via MHA and FFN), and "adds" its insight back onto the board for the next layer to read.

11.9 Common Misconceptions

1. **"LayerNorm acts across the batch."** That is BatchNorm (used in ResNets). LayerNorm normalizes across the d_m feature dimensions of a *single* token. The mean and variance are unique to that token independently of the batch or the sequence.

2. **"The FFN mixes information between words."** Absolutely not. The FFN matrix multiplication is applied to the T dimension independently. Only the Attention layer allows tokens t_1 and t_2 to exchange data. The FFN is purely intra-token processing.

11.10 Exercises

Warmup

1. State the difference between Pre-Norm and Post-Norm equations.

2. In a Transformer with $d_m = 1024$ and an FFN expansion of 4, what are the exact dimensions of the two FFN weight matrices?

Derivation

1. Assume you stack 100 Pre-Norm Transformer blocks: $\mathbf{x}_i = \mathbf{x}_{i-1} + f_i(\mathbf{x}_{i-1})$. Write the equation for $\mathbf{x}_{100}$ in terms of $\mathbf{x}_0$ and the sum of the sublayer outputs. Then derive $\frac{\partial \mathbf{x}_{100}}{\partial \mathbf{x}_0}$ to clearly prove why gradients do not vanish.

Implementation

1. [**Prepares for CS336 A1**] Implement a PyTorch `nn.Module` named `TransformerBlock`. It should take `x` (shape B, T, d_m), pass it through your simulated MHA and FFN modules using standard PyTorch `nn.LayerNorm`, and output the residual-summed tensor. Use no biases in the linear layers.

Open-Ended

1. If the residual stream $\mathbf{x}$ allows the model to bypass layers entirely by setting sublayer outputs to 0, what prevents the model from lazily doing nothing and just passing the raw input embeddings all the way to the output?

11.11 Further Reading

- **he2016**: Deep Residual Learning for Image Recognition. The ResNet paper that introduced the $\mathbf{x} + f(\mathbf{x})$ mathematical trick.
- **xiong2020**: On Layer Normalization in the Transformer Architecture. A rigorous mathematical analysis of why Post-Norm fails and why Pre-Norm stabilizes deep training.

Chapter 12

Position Embeddings: RoPE and ALiBi

Key Insight

After reading this chapter, you will be able to:

1. Explain why the vanilla self-attention mechanism is inherently permutation equivariant.
2. Differentiate between absolute positioning (sinusoidal, learned) and relative positioning.
3. Mathematically derive Rotary Position Embeddings (RoPE) and explain how they inject relative positioning via phase shifts.
4. Describe ALiBi and how it enables length extrapolation without positional parameters.

12.1 Big Picture

Self-attention computes the interaction between tokens by taking the dot product of their Queries and Keys ($\frac{\mathbf{QK}^\top}{\sqrt{d_k}}$). However, a dot product between two vectors is purely a function of the content of the vectors. It has zero knowledge of *where* the vectors came from in the sequence.

If you scramble the words in a sentence, the attention mechanism will compute the exact same similarity scores, just in a scrambled order. Self-attention treats language not as a sequence, but as an unordered bag of words. This is fatal: "The dog bit the man" and "The man bit the dog" mean highly different things.

To fix this, we must inject positional information into the vectors so the model can distinguish between "the" at position 1 and "the" at position 4. How we inject this information has evolved remarkably since 2017.

12.2 Formal Setup and Notation

Let $\mathbf{x}_m \in \mathbb{R}^d$ be the embedding of the token at position m, where $m \in \{1, \ldots, T\}$. Let $f_q(\mathbf{x}_m, m)$ and $f_k(\mathbf{x}_n, n)$ be functions that incorporate positional information into the queries and keys.

We want the attention score between position m and position n to be a function of both the word embeddings $(\mathbf{x}_m, \mathbf{x}_n)$ and their relative distance $m - n$:

$$\langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle = g(\mathbf{x}_m, \mathbf{x}_n, m - n) \tag{12.1}$$

12.3 Absolute Position Embeddings

The original Transformer simply added a position vector to the token embedding at the very input of the model:

$$\mathbf{x}'_m = \mathbf{x}_m + \mathbf{p}_m \tag{12.2}$$

where $\mathbf{p}_m \in \mathbb{R}^d$ is the position embedding for position m.

12.3.1 Sinusoidal Vectors

Vaswani et al. hand-crafted $\mathbf{p}_m$ using sine and cosine waves of varying frequencies.

$$p_{m,2i} = \sin(m/10000^{2i/d}) \tag{12.3}$$

$$p_{m,2i+1} = \cos(m/10000^{2i/d}) \tag{12.4}$$

Because trigonometric identities allow $\sin(m + k)$ to be expressed as a linear combination of $\sin(m)$ and $\cos(m)$, the hope was the model could easily learn relative differences.

12.3.2 Learned Absolute Vectors

Later models like GPT-2 and GPT-3 abandoned the fixed sinusoids. They simply initialized $\mathbf{p}_m$ as a trainable parameter matrix of shape $(T_{\max}, d)$. The model learns what position 1 "means" during training. **The Flaw:** The model can never process a sequence longer than $T_{\max}$ because it has no vector for position $T_{\max} + 1$.

12.4 Rotary Position Embeddings (RoPE)

RoPE (Su et al., 2021) is the standard in modern LLMs (LLaMA, DeepSeek, Mistral). It is mathematically elegant because it acts multiplicatively rather than additively, and applies directly to the Queries and Keys at every layer.

Instead of adding a vector, RoPE *rotates* the vector by an angle proportional to its position m. To rotate a 2D vector (x_1, x_2) by angle $m\theta$, we apply the 2D rotation matrix:

$$\mathbf{R}_{m\theta} = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix} \tag{12.5}$$

For a d-dimensional vector, we group the dimensions into $d/2$ pairs and apply a separate 2D rotation to each pair, with frequencies $\theta_i = 10000^{-2i/d}$. Let $\mathbf{R}\Theta, m \in \mathbb{R}^{d\times d}$ be the block-diagonal rotation matrix for position m.

$$f_q(\mathbf{x}_m, m) = \mathbf{R}_{\Theta,m}\mathbf{W}_Q\mathbf{x}_m, \quad f_k(\mathbf{x}_n, n) = \mathbf{R}_{\Theta,n}\mathbf{W}_K\mathbf{x}_n \tag{12.6}$$

Why does this work? Rotation matrices are orthogonal ($\mathbf{R}^\mathsf{T} = \mathbf{R}^{-1}$). The dot product between a rotated query at m and a rotated key at n simplifies beautifully:

$$\begin{aligned} \langle \mathbf{R}_m\mathbf{q}_m, \mathbf{R}_n\mathbf{k}_n \rangle &= (\mathbf{R}_m\mathbf{q}_m)^\mathsf{T}(\mathbf{R}_n\mathbf{k}_n) && (12.7) \\ &= \mathbf{q}_m^\mathsf{T}\mathbf{R}_m^\mathsf{T}\mathbf{R}_n\mathbf{k}_n && (12.8) \\ &= \mathbf{q}_m^\mathsf{T}\mathbf{R}_{n-m}\mathbf{k}_n && (12.9) \end{aligned}$$

The attention score relies *only* on the relative distance $n - m$. We achieved perfect relative positioning through absolute rotations!

12.5 ALiBi: Attention with Linear Biases

What if we don't modify the vectors at all, but modify the attention score directly? ALiBi (Press et al., 2021) directly subtracts a linear penalty based on the distance between the query and key:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\mathsf{T}}{\sqrt{d_k}} - \gamma|m - n|\right)\mathbf{V} \tag{12.10}$$

Here, γ is a fixed scalar that differs for each attention head. The further apart two tokens are, the more heavily their attention score is penalized.

Length Extrapolation: ALiBi allows models trained on 1,024 tokens to evaluate flawlessly on 2,048 tokens without fine-tuning, because there are no positional parameters bounding the sequence.

12.6 Worked Example: Implementing RoPE efficiently

Materializing the full $d \times d$ block-diagonal matrix $\mathbf{R}_m$ in memory is a catastrophic waste of VRAM, because it consists almost entirely of zeroes.

In PyTorch, we implement RoPE using complex numbers or interleaved vector multiplication. Given vector $\mathbf{q} = [q_0, q_1, q_2, q_3]$, we split it: $\mathbf{q}_1 = [q_0, q_2]$, $\mathbf{q}_2 = [q_1, q_3]$. The rotated vector is computed physically as:

$$\mathbf{q}_{\text{rotated}} = \begin{bmatrix} q_0 \cos(m\theta_0) - q_1 \sin(m\theta_0) \\ q_1 \cos(m\theta_0) + q_0 \sin(m\theta_0) \\ \vdots \end{bmatrix} \tag{12.11}$$

This uses element-wise multiplication ($\odot$) and takes $O(d)$ speed rather than $O(d^2)$ matrix multiplication.

12.7 Systems and Implementation Consequences

Because RoPE is applied inside every attention layer *after* the $\mathbf{Q}, \mathbf{K}$ projections, it fundamentally ties the sequence length T into the heart of the inner loop. When doing autoregressive generation (inference), we process just one new token at position $T + 1$. We must rotate it by $(T + 1)\theta$. We do not need to re-rotate the history in the KV-cache, because RoPE is computed once per token and cached.

Neuroscience Analogy: Grid Cells and Spatial Frequencies

Where the analogy helps. RoPE uses varying frequencies θ_i to encode position. High frequencies rotate quickly and provide fine-grained local position. Low frequencies rotate slowly and provide global context. This is fundamentally identical to "Grid Cells" in the mammalian entorhinal cortex, which fire in hexagonal patterns of varying spatial frequencies to give an animal a cognitive map of its environment.

Where the analogy breaks. Animals constantly update their position integration in a true recurrent loop over time using velocity. A Transformer has simultaneous access to all spatial coordinates through an instantaneous array lookup.

What intuition to keep. Position is not a single number (like index 4), but a high-dimensional vector combining multiple repeating oscillations that allow precise localization at multiple scales.

12.8 Common Misconceptions

1. **"RoPE models can extrapolate to infinite lengths."** While RoPE represents relative distances flawlessly in theory, the model never saw distances $> T_{\text{train}}$ during training. If confronted with a distance of 8,000, the attention weights will likely go uncalibrated. Length extrapolation with RoPE requires tricks like YaRN or Position Interpolation.

2. **"Position embeddings are added at every layer."** Absolute position embeddings are added only once, directly to the lowest input tokens. RoPE, however, is applied at every single attention layer specifically to the Q and K vectors.

12.9 Exercises

Warmup

1. Why do we not apply RoPE to the Value ($\mathbf{V}$) vectors?

2. In ALiBi, if $m = n$ (a token attends to itself), what is the penalty applied to the score?

Derivation

1. Using the sum/difference trigonometric identities ($\cos(A - B) = \cos A \cos B + \sin A \sin B$), explicitly derive $\mathbf{q}_m^\top \mathbf{k}_n$ for a 2D space under RoPE. Prove that the result is independent of absolute m or n and only depends on $m - n$.

Implementation

1. [**Prepares for CS336 A1**] Implement a PyTorch function `apply_rope(q, freqs)` that takes queries of shape (B, T, h, d_k) and pre-computed cosine/sine frequencies of shape (T, d_k), and applies the rotation efficiently.

Open-Ended

1. If RoPE explicitly embeds sequence distances $m - n$, how does this complicate efforts to use a 1D language model structure to process 2D images (like in Vision Transformers)? How might you adapt RoPE for a 2D grid?

12.10 Further Reading

- **su2021**: RoFormer: Enhanced Transformer with Rotary Position Embedding.
- **press2021**: Train Short, Test Long: Attention with Linear Biases (ALiBi).

Chapter 13

Initialization and Pre-training Setup

Key Insight

After reading this chapter, you will be able to:

1. Initialize weight matrices properly to ensure gradients don't explode or vanish before training even begins.
2. Understand the derivation of Xavier and Kaiming initialization.
3. Explain standard scaling trick associated with the residual connections in deep networks ($1/\sqrt{2N}$).
4. Structure a standard Transformer pre-training loop.

13.1 Big Picture

Imagine putting a sports car engine in a chassis, locking the steering wheel hard right, and pressing the gas. Even if to the components are mathematically perfect, the car will immediately crash.

This is what happens when you instantiate a neural network with improper weights. If weights are too large, activations explode to infinity and gradients become NaN (Not a Number) within the first five steps. If weights are too small, activations shrink to zero, and the model never learns.

Building a Transformer requires intense care in *how* the numbers begin. A properly initialized model will drop its loss significantly in the very first step. This chapter outlines the exact rigorous statistical standards required to initialize and assemble an LLM.

13.2 Formal Setup and Notation

Consider a simple linear layer $\mathbf{y} = \mathbf{W}\mathbf{x}$, where:

- Input $\mathbf{x} \in \mathbb{R}^{n_{\text{in}}}$, whose elements have variance $\text{Var}(x)$.
- Output $\mathbf{y} \in \mathbb{R}^{n_{\text{out}}}$, whose elements have variance $\text{Var}(y)$.
- Weights $\mathbf{W} \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$, initialized from a distribution with variance $\text{Var}(w)$.

Assuming x_i and W_{ij} are independent and have a mean of 0, the variance of an output element $y_i = \sum_{j=1}^{n_{\text{in}}} W_{ij} x_j$ is:

$$\text{Var}(y_i) = n_{\text{in}} \cdot \text{Var}(w) \cdot \text{Var}(x) \tag{13.1}$$

13.3 Variance Preservation

To ensure signal flows cleanly, we want the variance of the output to equal the variance of the input: $\text{Var}(y) = \text{Var}(x)$.

Plugging this into Equation Equation (13.1):

$$1 = n_{\text{in}} \cdot \text{Var}(w) \implies \text{Var}(w) = \frac{1}{n_{\text{in}}} \tag{13.2}$$

If we draw our weights from a Normal distribution $\mathcal{N}(0, \sigma^2)$, we must set the standard deviation to:

$$\sigma = \frac{1}{\sqrt{n_{\text{in}}}} \tag{13.3}$$

This is the foundational math behind **Xavier (Glorot) Initialization**.

If we place a ReLU non-linearity after the linear layer, the ReLU zeroes out exactly half of the distribution. This cuts the variance in half. To compensate, we must double the variance of the weights:

$$\sigma = \sqrt{\frac{2}{n_{\text{in}}}} \tag{13.4}$$

This is **Kaiming (He) Initialization**, standard for deep CNNs with ReLU.

13.4 Transformer-Specific Initialization

Transformers have several unique architectural properties that require specialized scaling:

13.4.1 Standard Deviation

Most weights in the Transformer (Query, Key, Value matrices, FFN matrices) are initialized from a normal distribution $\mathcal{N}(0, \sigma^2)$ with $\sigma = 0.02$. Why 0.02? For a model dimension $d_m = 4096$,

$1/\sqrt{4096} \approx 0.015$. A value of 0.02 is a robust, empirical approximation for the Xavier bounds that stuck in the literature (e.g., GPT-2).

13.4.2 Residual Deep Scaling

In a pre-norm architecture, the residual stream $\mathbf{x}$ receives an addition from every sublayer. If there are N blocks, there are $2N$ additions (one for MHA, one for FFN).

Because variances sum ($\text{Var}(A + B) = \text{Var}(A) + \text{Var}(B)$ for independent variables), the variance of the residual stream grows substantially as it passes through the network. To prevent deep network collapse, we scale the initialization standard deviation of the final projection matrices (the W_O in attention, and the final combined matrix in the FFN) by a factor inversely proportional to the depth:

$$\sigma = \frac{0.02}{\sqrt{2N}} \tag{13.5}$$

13.5 The Training Loop Setup

Building a robust training script requires carefully orchestrating the dataloader and optimizer. A standard pre-training step functions as follows:

1. Sample a batch $\mathbf{X}$ of shape (B, T) and a target $\mathbf{Y}$ shifted by one.
2. Pass $\mathbf{X}$ through the embedding layer, applying RoPE incrementally at each of the N layers, resolving the logits matrix of shape (B, T, V).
3. Reshape logits to $(B \times T, V)$ and targets to $(B \times T)$ to compute the Cross-Entropy loss.
4. Trigger `loss.backward()` to compute gradients.
5. **Gradient Clipping**: Clamp all gradients to a maximum $L2$ norm (typically 1.0) to prevent rogue data patches from creating massive gradients that destroy the weights.
6. Call `optimizer.step()` using AdamW, usually paired with a learning rate scheduler featuring a linear warmup and a cosine decay.

13.6 Worked Example: Checking Initialization in PyTorch

If you build an LLM cleanly, you should manually initialize weights rather than trusting PyTorch defaults (which use an atypical Kaiming Uniform derived from a specific heuristic).

```python
def init_weights(module):
    if isinstance(module, nn.Linear):
        # Base init for dense weights
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
```

```
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        # Embeddings need slightly higher variance to cover space
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

# Specific override for residual projections
# Suppose model has config.n_layers
std_res = 0.02 / math.sqrt(2 * config.n_layers)
for name, p in model.named_parameters():
    if name.endswith('c_proj.weight') or name.endswith('w2.weight'):
        torch.nn.init.normal_(p, mean=0.0, std=std_res)
```

13.7 Systems and Implementation Consequences

Precision is disastrously tight at initialization if using half-precision (FP16). Gradients are extremely small and can easily underflow to exactly 0.0 in FP16. We mitigate this using a **Gradient Scaler** (multiplying the loss by a massive number like 2^{16} before backprop, so gradients are large enough to be represented in FP16, then dividing the gradients down when applying the optimizer). In modern setups, BFloat16 natively solves this underflow issue, which is why it is strictly preferred over FP16.

Neuroscience Analogy: Critical Periods and Wiring Diagram Initialization

Where the analogy helps. At birth, the mammalian brain does not start with completely random synapses. Genetic encoding ensures a tightly constrained initial "wiring diagram" with specific connection probabilities and variances. If this initialization is biologically derailed, the network fails to learn properly (as seen in neurodevelopmental disorders). In ML, statistical variance rules provide this necessary starting template.

Where the analogy breaks. Biological brains dynamically prune redundant connections over time (synaptic pruning). An LLM's architecture is completely static; the matrix dimensions never shrink.

What intuition to keep. A model's ability to learn is overwhelmingly dependent on its state at time $t = 0$. Good initialization provides a smooth fitness landscape; bad initialization guarantees failure regardless of how much compute you have.

13.8 Common Misconceptions

1. **"PyTorch handles initialization automatically."** PyTorch's default 'nn.Linear' initialization uses a Kaiming uniform bound $U(-\sqrt{k}, \sqrt{k})$. It does not match the Gaussian

$\mathcal{N}(0, 0.02)$ initialization universally used in LLM papers (like LLaMA).

2. **"Biases should be initialized randomly."** Biases should almost always be initialized precisely to 0.0.

13.9 Exercises

Warmup

1. If $N = 18$, what is the scaling factor $\frac{1}{\sqrt{2N}}$? How does this change the standard deviation of the final layer?

Derivation

1. Show that if X and Y are independent random variables with zero mean, $\text{Var}(XY) = \text{Var}(X)\text{Var}(Y)$. Use this to rigorously prove Equation Equation (13.1).

Implementation

1. [**Prepares for CS336 A1**] Implement the `init_weights` hook on your `TransformerBlock` module. Print the standard deviation of your `c_proj.weight` tensor before and after your hook runs to prove it worked.

13.10 Further Reading

- **glorot2010**: Understanding the difficulty of training deep feedforward neural networks (The Xavier Paper).
- **radford2019**: Language Models are Unsupervised Multitask Learners (GPT-2). Specifically for the $\frac{1}{\sqrt{2N}}$ residual scaling trick notation.

Chapter 14

Mixture of Experts

Key Insight

After reading this chapter, you will be able to:

1. Explain why Mixture of Experts (MoE) decouples total model parameters from per-token computation, and why this matters for scaling.
2. Formalize the gating function, top-k routing, and capacity factor, and work through the math of a single MoE forward pass.
3. Identify and analyze the failure modes of sparse routing: expert collapse, load imbalance, and token dropping, and explain the auxiliary losses designed to mitigate them.
4. Describe how expert parallelism distributes experts across devices, including the all-to-all communication pattern and its systems cost.
5. Compare the training and inference tradeoffs of sparse MoE models (Switch, GShard, Mixtral, DeepSeek) against dense baselines.

14.1 Big Picture

In Chapters 9 and 10, we built the standard Transformer block: attention followed by a feedforward network (FFN). The FFN contains the majority of the model's parameters. In GPT-3 (175B), roughly two-thirds of all parameters live inside the FFN layers. These parameters encode the model's "knowledge"—factual associations, linguistic patterns, and world models compressed from the training data.

A natural question arises: *can we scale up the model's knowledge capacity without proportionally scaling the computation required for every single token?*

The answer is **Mixture of Experts (MoE)**. The core idea is deceptively simple: instead of one large FFN, use E smaller FFNs (called **experts**) and a learned **router** (or **gating network**) that selects which experts process each token. If the router activates only $k \ll E$ experts per token, then:

- The **total parameter count** scales with E (more experts = more knowledge capacity).
- The **per-token FLOPs** scale with k (only a few experts fire per token).

This is *conditional computation*: different tokens take different computational paths through the network. A token about chemistry might activate expert 3 and expert 7; a token about syntax might activate expert 1 and expert 5. The model allocates its capacity dynamically.

MoE is not a niche research curiosity. It is a core production technique:

- Mixtral 8×7B uses 8 experts with top-2 routing, achieving "7B-class" inference cost with "47B-class" quality.
- DeepSeek-V3 uses 256 routed experts with top-8 routing plus 1 shared expert, totaling 671B parameters but activating only 37B per token.
- Google's Switch Transformer demonstrated that even top-1 routing (one expert per token) can work if the auxiliary losses are designed correctly.

CS336 dedicates an entire lecture (L4) to MoE because it represents a fundamental design axis: the tradeoff between *parameter capacity* and *active computation*.

14.2 Formal Setup and Notation

We work within a single Transformer block. Recall the standard dense FFN from Chapter 10:

$$\mathrm{FFN}(\mathbf{x}) = \mathbf{W}_2 \cdot \sigma(\mathbf{W}_1 \mathbf{x}) \tag{14.1}$$

where $\mathbf{x} \in \mathbb{R}^d$ is the hidden state for a single token, $\mathbf{W}_1 \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$, and σ is a nonlinearity (typically SwiGLU in modern architectures).

An **MoE layer** replaces this single FFN with:

- E expert networks $\{f_1, f_2, \ldots, f_E\}$, each an independent FFN with its own parameters.
- A **router** (gating network) $G : \mathbb{R}^d \to \mathbb{R}^E$ that produces a score for each expert given an input token.

We define:

$$T = \text{number of tokens in the batch} \tag{14.2}$$

$$E = \text{number of experts} \tag{14.3}$$

$$k = \text{number of experts activated per token (top-}k\text{)} \tag{14.4}$$

$$C = \text{expert capacity (max tokens an expert can process)} \tag{14.5}$$

$$d = \text{model dimension} \tag{14.6}$$

$$d_{\text{ff}} = \text{FFN hidden dimension per expert} \tag{14.7}$$

14.3 The Router: From Dense to Sparse

14.3.1 Gating Function

The router computes a score vector over all experts for each token $\mathbf{x}$:

$$\mathbf{s}(\mathbf{x}) = \mathbf{W}_g \mathbf{x} \tag{14.8}$$

where $\mathbf{W}_g \in \mathbb{R}^{E \times d}$ is the gating weight matrix. Each entry $s_i(\mathbf{x})$ represents how relevant expert i is for token $\mathbf{x}$.

14.3.2 Top-k Selection

We select the top-k experts by score and apply softmax *only over the selected experts* to get the gating weights:

$$g_i(\mathbf{x}) = \begin{cases} \frac{e^{s_i(\mathbf{x})}}{\sum_{j \in \text{TopK}(\mathbf{s},k)} e^{s_j(\mathbf{x})}} & \text{if } i \in \text{TopK}(\mathbf{s}(\mathbf{x}), k) \\ 0 & \text{otherwise} \end{cases} \tag{14.9}$$

The output of the MoE layer is the weighted combination of the selected experts:

$$\text{MoE}(\mathbf{x}) = \sum_{i=1}^{E} g_i(\mathbf{x}) \cdot f_i(\mathbf{x}) \tag{14.10}$$

Because $g_i = 0$ for all but k experts, only k expert forward passes are computed. This is where the computational saving comes from.

14.3.3 Top-1 vs. Top-2 Routing

Top-1 (Switch Transformer): Each token goes to exactly one expert. Maximally sparse. Simplest dispatch logic. But the gradient signal through the router is noisier because each token only "sees" one expert.

Top-2 (GShard, Mixtral): Each token goes to two experts and their outputs are blended by the gating weights. Smoother gradients, better quality, but double the computation per token compared to top-1.

Modern systems like DeepSeek-V3 use higher k (top-8 out of 256 experts) combined with a **shared expert** that processes every token regardless of routing, providing a stable baseline representation.

14.4 Capacity Factor and Token Dropping

14.4.1 The Load Imbalance Problem

If the router assigns 90% of tokens to expert 3 and 1% to expert 7, we have a catastrophic load imbalance. Expert 3 is overwhelmed; expert 7 is wasted. In a distributed setting where each expert lives on a different device, one GPU is overloaded while others sit idle.

14.4.2 Capacity Factor

To manage this, we define the **expert capacity**:

$$C = \left\lceil \frac{T}{E} \cdot c_f \right\rceil \tag{14.11}$$

where c_f is the **capacity factor**, typically $c_f \in [1.0, 1.5]$. With $c_f = 1.0$, each expert can handle exactly its "fair share" of tokens (T/E). With $c_f = 1.25$, each expert gets a 25% buffer.

14.4.3 Token Dropping

When an expert's buffer is full (C tokens already assigned), additional tokens routed to that expert are **dropped**: they skip the expert computation entirely and their representation passes through unchanged via the residual connection.

Token dropping has real consequences:

- Dropped tokens lose the benefit of the FFN computation at that layer.
- The model may develop blind spots for certain token types that consistently overflow popular experts.
- During training, dropped tokens still produce gradients for the router (encouraging it to spread load), but the expert parameters miss those training signals.

14.5 Auxiliary Load Balancing Loss

The gating function is differentiable (softmax over top-k scores), so the router learns via gradient descent. But left to its own devices, the router tends to collapse: it discovers that a few experts are "good enough" and routes almost all tokens to them. This is a positive feedback loop—popular experts get more training signal, become even better, attract even more tokens.

To prevent this, we add an **auxiliary load balancing loss** to the training objective.

14.5.1 The Switch Transformer Formulation

Define two quantities over a batch of T tokens:

$$f_i = \frac{1}{T} \sum_{t=1}^{T} \mathbf{1}[\text{token } t \text{ is routed to expert } i] \quad \text{(fraction of tokens dispatched to expert } i\text{)} \tag{14.12}$$

$$p_i = \frac{1}{T} \sum_{t=1}^{T} g_i(\mathbf{x}_t) \quad \text{(mean gating probability for expert } i\text{)} \tag{14.13}$$

The auxiliary loss is:

$$\mathcal{L}_{\text{aux}} = \alpha \cdot E \cdot \sum_{i=1}^{E} f_i \cdot p_i \tag{14.14}$$

where α is a small coefficient (typically $\alpha = 0.01$).

Why does this work? The product $f_i \cdot p_i$ is minimized when both f_i and p_i are uniform across experts (i.e., $f_i = 1/E$ and $p_i = 1/E$ for all i). If expert 3 is overloaded ($f_3 \gg 1/E$) and has high average probability ($p_3 \gg 1/E$), the product $f_3 \cdot p_3$ is large, and the loss penalizes this imbalance. The factor E normalizes the loss magnitude.

14.5.2 Aux-Loss-Free Approaches

Recent work (DeepSeek-V3) has explored removing the auxiliary loss entirely, replacing it with per-expert bias terms that are adjusted dynamically during training to maintain balance. The motivation: the auxiliary loss can conflict with the primary language modeling objective, forcing suboptimal routing for the sake of balance. Bias-based approaches decouple load balancing from the gradient signal.

14.6 Expert Collapse and Training Instability

MoE training is notoriously fragile. Common failure modes include:

1. **Expert collapse**: One or more experts receive negligible traffic and their parameters stop updating. Once collapsed, an expert is effectively dead—the router has learned to avoid it, and the expert has no gradient signal to recover.

2. **Router collapse**: The router learns a near-deterministic mapping (always picks the same experts regardless of input), losing the benefits of conditional computation.

3. **Training spikes**: MoE models exhibit more frequent loss spikes during training than dense models. When the router suddenly shifts its assignments (a "routing earthquake"), entire batches of tokens hit unfamiliar experts, causing loss spikes.

4. **z-loss**: To stabilize training, some implementations add a penalty on the magnitude of the router logits: $\mathcal{L}_z = \frac{1}{T}\sum_t \left(\log \sum_i e^{s_i(\mathbf{x}_t)}\right)^2$. This prevents the logits from becoming too large, which would make the softmax too sharp and the routing too deterministic.

14.7 Dispatch and Combine: Implementation

In practice, the MoE forward pass involves two key operations:

Dispatch: Reorganize tokens from their natural batch order into per-expert groups. Token t (originally at position t in the batch) must be sent to its selected experts. This creates E variable-length buffers, each containing the tokens assigned to that expert (up to capacity C).

Expert computation: Each expert processes its assigned tokens independently. Because experts are independent FFNs, this step is embarrassingly parallel.

Combine: After expert computation, route the outputs back to their original positions in the batch and weight them by the gating coefficients $g_i(\mathbf{x}_t)$.

In a single-device implementation, dispatch and combine are index-gathering operations. In a multi-device setting, they become **all-to-all communication** operations.

14.8 All-to-All Communication

When experts are distributed across P devices (expert parallelism), the dispatch step requires an **all-to-all collective**: each device sends a subset of its local tokens to every other device (wherever the selected experts reside), and receives tokens from all other devices for its local experts.

This is fundamentally different from the all-reduce used in data parallelism:

- **All-reduce** (data parallelism): Every device sends the same quantity of data (gradient tensors). Communication volume is predictable.

- **All-to-all** (expert parallelism): Each device sends a *variable* amount of data depending on routing decisions. Communication is data-dependent and potentially imbalanced.

Two all-to-all operations are required per MoE layer: one for dispatch (tokens $\rightarrow$ experts) and one for combine (expert outputs $\rightarrow$ original positions). For a model with L_{MoE} MoE layers, this is $2L_{\text{MoE}}$ all-to-all operations per forward pass.

14.9 Expert Parallelism

Expert parallelism (EP) distributes the E experts across P devices, with each device hosting E/P experts. It is orthogonal to other parallelism strategies:

- **Data parallelism** (DP): Replicates the full model; splits the batch. Communication: all-reduce on gradients.
- **Tensor parallelism** (TP): Splits individual weight matrices across devices. Communication: all-reduce on activations.
- **Pipeline parallelism** (PP): Splits layers across devices. Communication: point-to-point activation transfers.
- **Expert parallelism** (EP): Splits experts across devices. Communication: all-to-all on tokens.

In practice, these are composed. DeepSeek-V3 uses a combination of DP + TP + PP + EP. The key constraint is that EP requires high-bandwidth interconnect between the expert-parallel group because every MoE layer triggers all-to-all communication.

14.10 Worked Example: Routing 8 Tokens Through 4 Experts

Consider a batch of $T = 8$ tokens, $E = 4$ experts, top-$k = 2$ routing, and capacity factor $c_f = 1.25$.

Step 1: Compute expert capacity.

$$C = \left\lceil \frac{8}{4} \times 1.25 \right\rceil = \lceil 2.5 \rceil = 3$$

Each expert can handle at most 3 tokens.

Step 2: Router scores. Suppose the router produces these scores (after softmax over top-2):

Token	g_1	g_2	g_3	g_4
t_1	**0.7**	0	**0.3**	0
t_2	0	**0.6**	**0.4**	0
t_3	**0.5**	**0.5**	0	0
t_4	0	0	**0.8**	**0.2**
t_5	**0.9**	0	**0.1**	0
t_6	**0.4**	**0.6**	0	0
t_7	0	0	**0.3**	**0.7**
t_8	0	**0.5**	**0.5**	0

Step 3: Count dispatches per expert.

- Expert 1: tokens t_1, t_3, t_5, t_6 (4 tokens) $\rightarrow$ exceeds capacity $C = 3$!
- Expert 2: tokens t_2, t_3, t_6, t_8 (4 tokens) $\rightarrow$ exceeds capacity!
- Expert 3: tokens $t_1, t_2, t_4, t_5, t_7, t_8$ (6 tokens!) $\rightarrow$ exceeds capacity!
- Expert 4: tokens t_4, t_7 (2 tokens) $\rightarrow$ within capacity.

Step 4: Apply capacity constraints (process in order, drop overflows).

- Expert 1 accepts t_1, t_3, t_5; **drops** t_6's request for Expert 1.
- Expert 2 accepts t_2, t_3, t_6; **drops** t_8's request for Expert 2.
- Expert 3 accepts t_1, t_2, t_4; **drops** t_5, t_7, t_8.

Token t_8 had both selected experts overflow. It passes through the residual connection with no expert computation—this is a dropped token.

Step 5: Combine. For token t_1 (routed to experts 1 and 3, both accepted):

$$\text{output}(t_1) = 0.7 \cdot f_1(\mathbf{x}_{t_1}) + 0.3 \cdot f_3(\mathbf{x}_{t_1})$$

For token t_8 (both experts overflowed):

$$\text{output}(t_8) = \mathbf{x}_{t_8} \quad \text{(residual only)}$$

14.11 Systems and Implementation Consequences

14.11.1 Training: Memory and Throughput

MoE models have a paradoxical systems profile:

- **FLOPs per token**: Roughly k/E of the dense equivalent. A 47B MoE with $k = 2, E = 8$ uses about the same FLOPs as a 12B dense model.
- **Memory**: The *full* E expert parameter sets must be stored, even though only k are active per token. A 47B parameter MoE requires 47B parameters' worth of memory, not 12B. Optimizer states (Adam moments) scale with total parameters, not active parameters.
- **Communication**: Each MoE layer requires two all-to-all operations. For $L_{\text{MoE}} = 32$ MoE layers, that is 64 all-to-all calls per forward pass. This can dominate training time on clusters with limited inter-node bandwidth.
- **Throughput**: MoE training throughput (tokens/second) is typically 30–60% of a compute-equivalent dense model, because the all-to-all overhead and load imbalance reduce GPU utilization.

14.11.2 Inference: Latency, Throughput, and Cache

Inference with MoE models creates unique challenges:

- **Latency**: During autoregressive generation, each token must be routed to its experts. If experts reside on different devices, this requires network communication for every single generated token.
- **Throughput**: Large batch serving can amortize the all-to-all cost. But small-batch (e.g., single-user) inference hits the worst case: full all-to-all latency for one token.
- **KV cache**: The KV cache is shared across all experts (it belongs to the attention layers, which are dense). MoE does not change KV cache size.
- **Expert memory**: All E experts must be resident in memory even though only k are used per token. This makes MoE models much harder to serve on memory-limited hardware.
- **Expert offloading**: Some inference systems offload inactive experts to CPU and load them on-demand, trading latency for memory. This is practical for moderate E but breaks down for $E = 256$.

14.11.3 Sparse vs. Dense Tradeoffs

The fundamental tradeoff:

Property	**Dense**	**MoE (Sparse)**
Parameters	N	$\sim E \cdot N/k$
Active FLOPs/token	N	$\sim N$ (same as small dense)
Memory (weights)	N	$E \cdot N/k$ (all experts stored)
Memory (optimizer)	$3N$ (Adam)	$3 \cdot E \cdot N/k$
Communication	All-reduce	All-reduce + All-to-all
Training stability	Stable	Requires aux loss, z-loss
Quality per FLOP	Baseline	2–4× better
Serving complexity	Simple	Expert routing, load balancing

The headline result is that MoE models achieve the quality of a much larger dense model at the compute cost of a much smaller one. But this comes at the price of higher memory, more complex communication, less stable training, and more complex serving infrastructure.

Common Pitfall

"More parameters" ≠ "more computation per token." A 671B MoE model that activates 37B parameters per token is *not* doing 671B parameters' worth of work on each token. It is doing roughly 37B parameters' worth of computation, but drawing on a much larger pool of stored knowledge. Confusing total parameters with active parameters is one of the most common errors when comparing MoE and dense models.

14.12 The MoE Landscape

14.12.1 Sparsely-Gated MoE (Shazeer et al., 2017)

The foundational paper [5] demonstrated that a learned gating network could route tokens to a subset of experts, achieving dramatic improvements in translation quality. Used top-2 routing with a noisy gating mechanism (adding Gaussian noise to the logits before top-k selection) to encourage exploration. Introduced the importance-based auxiliary loss.

14.12.2 GShard (Lepikhin et al., 2020)

GShard [6] scaled MoE to 600B parameters across 2048 TPUs. Its key contributions:

- Formalized the **capacity factor** c_f and token dropping.
- Introduced **random routing** for the second expert in top-2: the second expert is sampled proportionally to its gating weight rather than deterministically selected. This reduces dispatch imbalance.
- Demonstrated **expert parallelism** as a practical parallelism strategy composable with data and model parallelism.

14.12.3 Switch Transformer (Fedus et al., 2022)

Switch Transformers [7] simplified MoE by using **top-1 routing**: each token goes to exactly one expert. This halves the compute per MoE layer compared to top-2, simplifies dispatch (no weighted combination), and still achieves strong results. The paper introduced the $f_i \cdot p_i$ auxiliary loss formulation shown in Equation (14.14) and carefully studied the interaction between capacity factor, batch size, and expert count.

14.12.4 Mixtral (Mistral, 2024)

Mixtral 8×7B [8] is a landmark open-source MoE model: 8 experts with top-2 routing at every layer. Total parameters: 47B. Active parameters: ∼13B. It matches or exceeds LLaMA-2 70B

on most benchmarks while requiring roughly 6× less compute per token. Mixtral demonstrated that MoE could be practical for open-source deployment, not just internal research.

14.12.5 DeepSeek-V2 and V3

DeepSeek-V2 [9] introduced two innovations:

- **Multi-head Latent Attention (MLA)**: compresses the KV cache via low-rank projection.
- **Fine-grained expert segmentation**: Instead of 8 large experts, DeepSeek uses 160+ small experts with top-6 routing, achieving better load balance and specialization.

DeepSeek-V3 [10] scaled this to 671B total parameters (37B active), 256 routed experts + 1 shared expert, and introduced an **aux-loss-free** [11] load balancing approach using per-expert bias terms adjusted via a simple sequential algorithm during training.

14.13 Fine-Tuning Risks

Fine-tuning MoE models carries specific dangers:

- **Expert forgetting**: If the fine-tuning data is narrow (e.g., only code), the router may collapse to routing everything to a few code-specialized experts, causing other experts to degrade.
- **Load imbalance amplification**: Fine-tuning datasets are typically much smaller and less diverse than pretraining data. The router may overfit to the narrow distribution, creating severe imbalance.
- **Auxiliary loss tuning**: The α coefficient for the auxiliary loss may need re-calibration during fine-tuning. Too small α allows collapse; too large α forces unnatural routing that hurts task performance.

14.14 MoE, Long Context, and Deployment Cost

MoE interacts with long-context and deployment in subtle ways:

- **Long context**: MoE does not change the attention mechanism, so KV cache scaling with sequence length is unchanged. However, longer sequences mean more tokens per batch, which actually *helps* MoE by providing better load balance across experts (more tokens → closer to uniform distribution by the law of large numbers).
- **Deployment cost**: The full expert parameter set must be in memory. A 671B MoE model requires the same weight storage as a 671B dense model, even though it uses only

5% of those parameters per token. This makes MoE models expensive to deploy on single machines and practically requires multi-GPU serving.

- **Quantization**: MoE models respond well to weight quantization (INT8, INT4) because each expert is a standard FFN. However, the router weights must typically remain in higher precision to maintain routing quality.

Neuroscience Analogy: Cortical Specialization and Sparse Activation

Where the analogy helps. The mammalian cortex exhibits *functional specialization*: the fusiform face area (FFA) preferentially responds to faces, Broca's area to language production, and area MT to visual motion. When you read text, your visual word form area activates strongly while your motor cortex for leg movements is relatively quiescent. This mirrors MoE routing: not all computational subunits are equally active for all inputs. Different inputs preferentially recruit different specialized modules, and the brain's total neural capacity far exceeds what is active at any given moment.

Where the analogy breaks. Crucially, the brain does not have an explicit top-k router that gates information flow with a learned weight matrix. Cortical specialization emerges from developmental wiring, experience-dependent plasticity, and anatomical connectivity—not from a differentiable softmax over a set of discrete modules. There is no "capacity factor" in the brain; neural populations can flexibly handle variable load through rate coding and population dynamics. The timescales of biological adaptation (milliseconds for neural dynamics, days to years for synaptic plasticity) bear no resemblance to the single-step gradient update that trains an MoE router.

What intuition to keep. The key insight is: *sparse, input-dependent activation of specialized subnetworks can be more efficient than dense, uniform computation.* Specialization enables capacity without proportional cost—but coordination and routing between specialized modules introduces its own overhead and failure modes, whether in brains or in Transformers.

14.15 Common Misconceptions

1. **"An MoE model with 671B parameters is 671B parameters smart."** No. On any given token, only ∼37B parameters are active. The model's per-token computational depth is comparable to a 37B dense model. The 671B figure represents stored capacity (like a library with many books), not active reasoning power (like reading speed).

2. **"MoE saves memory because fewer parameters are active."** Incorrect. All E expert parameter tensors and their Adam optimizer states must be stored in GPU memory. MoE saves *FLOPs*, not *memory*. In fact, MoE models require *more* total memory than a

compute-equivalent dense model.

3. **"Experts naturally specialize into interpretable topics like 'code' or 'math'."** This is sometimes true but often overstated. Empirical analysis of expert activation patterns shows that specialization is typically at the level of token-level syntactic patterns (punctuation, whitespace, certain morphological patterns) rather than clean semantic categories. A few experts may show domain affinity, but most exhibit mixed-domain activation.

4. **"MoE is just dropout applied to FFN layers."** Dropout randomly zeroes out neurons during training for regularization. MoE *selectively routes* tokens to entire expert FFNs based on a learned gating function. The selection is deterministic (given the input) and learned, not random. The purposes are entirely different: dropout prevents overfitting; MoE enables conditional computation for capacity scaling.

5. **"MoE models are harder to fine-tune, so you should always use dense models."** MoE fine-tuning has specific risks (expert forgetting, load collapse), but modern techniques (freezing the router, using lower learning rates for experts) make it practical. Mixtral and DeepSeek are routinely fine-tuned for specific applications.

14.16 Exercises

Warmup

1. In a model with $E = 16$ experts and top-$k = 2$ routing, what fraction of the expert parameters are active per token?

2. If the capacity factor is $c_f = 1.0$ and there are $T = 1024$ tokens in a batch with $E = 8$ experts, what is the maximum number of tokens each expert can process? What happens to the 129th token routed to a single expert?

3. Write the auxiliary loss $\mathcal{L}_{\text{aux}}$ value for a perfectly balanced routing scenario where $f_i = 1/E$ and $p_i = 1/E$ for all i, given $E = 8$ and $\alpha = 0.01$.

Derivation

1. Show that the auxiliary loss $\mathcal{L}_{\text{aux}} = \alpha \cdot E \cdot \sum_i f_i \cdot p_i$ is minimized (subject to $\sum_i f_i = k$ and $\sum_i p_i = k$) when $f_i = k/E$ and $p_i = k/E$ for all i. (Hint: use the AM-GM inequality or Cauchy-Schwarz.)

2. Derive the total FLOPs for a single MoE layer with E experts, each having hidden dimension d_{ff}, model dimension d, top-k routing, and T tokens. Compare this to a dense FFN with hidden dimension $E \cdot d_{\text{ff}}$ processing the same T tokens.

Implementation

1. Implement a simple MoE layer in PyTorch with $E = 4$ experts and top-2 routing. Your implementation should include:

   (a) A linear router $\mathbf{W}_g \in \mathbb{R}^{E \times d}$.

   (b) Top-k selection with softmax renormalization.

   (c) A capacity-limited dispatch buffer.

   (d) Weighted combination of expert outputs.

   Verify that exactly 2 experts fire per token and that tokens exceeding capacity are dropped.

Open-Ended

1. DeepSeek-V3 removed the auxiliary loss entirely, replacing it with adaptive bias terms. What are the theoretical advantages and risks of this approach compared to the Switch Transformer's $f_i \cdot p_i$ formulation? Under what training scenarios might aux-loss-free routing fail?

14.17 Further Reading

- Google [5]: *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.* The foundational paper introducing learned sparse routing for conditional computation. Establishes the mathematical framework and demonstrates scaling to 137B parameters.

- Google [7]: *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.* Simplifies MoE to top-1 routing, introduces the $f_i \cdot p_i$ auxiliary loss, and provides extensive ablations on capacity factor, expert count, and training stability.

- Lepikhin et al. [6]: *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding.* The systems paper for MoE at scale. Introduces expert parallelism, capacity factor, and random routing for top-2 gating.

- Mistral [8]: *Mixtral of Experts.* The first widely available open-source MoE model, demonstrating that sparse MoE provides excellent quality-per-FLOP tradeoffs in a practical, deployable package.

- "Deepseek V3" [10]: *DeepSeek-V3 Technical Report.* Pushes MoE to 671B parameters with 256 fine-grained experts and aux-loss-free load balancing, representing the current state of the art in sparse MoE design.

Part IV

Systems and Implementation

Chapter 15

Resource Accounting

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

15.1 Big Picture

TODO: Situate this chapter in the overall narrative.

15.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

15.3 Main Ideas

TODO: Core technical content.

15.4 Worked Example

TODO: Complete worked example.

15.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

15.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

15.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

15.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 16

GPUs, TPUs, and Memory Hierarchy

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

16.1 Big Picture

TODO: Situate this chapter in the overall narrative.

16.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

16.3 Main Ideas

TODO: Core technical content.

16.4 Worked Example

TODO: Complete worked example.

16.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

16.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

16.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

16.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 17

Kernels, CUDA, Triton, and FlashAttention

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

17.1 Big Picture

TODO: Situate this chapter in the overall narrative.

17.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

17.3 Main Ideas

TODO: Core technical content.

17.4 Worked Example

TODO: Complete worked example.

17.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

17.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

17.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

17.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 18

Parallelism and Distributed Training

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

18.1 Big Picture

TODO: Situate this chapter in the overall narrative.

18.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

18.3 Main Ideas

TODO: Core technical content.

18.4 Worked Example

TODO: Complete worked example.

18.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

18.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

18.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

18.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 19

Inference and Serving

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

19.1 Big Picture

TODO: Situate this chapter in the overall narrative.

19.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

19.3 Main Ideas

TODO: Core technical content.

19.4 Worked Example

TODO: Complete worked example.

19.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

19.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

19.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

19.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Part V

Scaling, Evaluation, and Recipes

Chapter 20

Scaling Laws

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

20.1 Big Picture

TODO: Situate this chapter in the overall narrative.

20.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

20.3 Main Ideas

TODO: Core technical content.

20.4 Worked Example

TODO: Complete worked example.

20.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

20.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

20.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

20.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 21

Model Recipes and Open-Model Case Studies

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

21.1 Big Picture

TODO: Situate this chapter in the overall narrative.

21.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

21.3 Main Ideas

TODO: Core technical content.

21.4 Worked Example

TODO: Complete worked example.

21.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

21.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

21.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

21.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 22

Evaluation

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

22.1 Big Picture

TODO: Situate this chapter in the overall narrative.

22.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

22.3 Main Ideas

TODO: Core technical content.

22.4 Worked Example

TODO: Complete worked example.

22.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

22.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

22.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

22.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Part VI

Data

Chapter 23

Data Sources, Crawling, and Cleaning

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

23.1 Big Picture

TODO: Situate this chapter in the overall narrative.

23.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

23.3 Main Ideas

TODO: Core technical content.

23.4 Worked Example

TODO: Complete worked example.

23.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

23.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

23.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

23.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 24

Filtering, Deduplication, and Data Mixing

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

24.1 Big Picture

TODO: Situate this chapter in the overall narrative.

24.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

24.3 Main Ideas

TODO: Core technical content.

24.4 Worked Example

TODO: Complete worked example.

24.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

24.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

24.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

24.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Part VII

Alignment and Reasoning

Chapter 25

Instruction Tuning and Preference Tuning

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

25.1 Big Picture

TODO: Situate this chapter in the overall narrative.

25.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

25.3 Main Ideas

TODO: Core technical content.

25.4 Worked Example

TODO: Complete worked example.

25.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

25.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

25.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

25.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 26

RLHF, DPO, GRPO, and Reasoning RL

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

26.1 Big Picture

TODO: Situate this chapter in the overall narrative.

26.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

26.3 Main Ideas

TODO: Core technical content.

26.4 Worked Example

TODO: Complete worked example.

26.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

26.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

26.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

26.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 27

Safety, Reward Hacking, and Deployment

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

27.1 Big Picture

TODO: Situate this chapter in the overall narrative.

27.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

27.3 Main Ideas

TODO: Core technical content.

27.4 Worked Example

TODO: Complete worked example.

27.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

27.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

27.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

27.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Appendices

Appendix A

Proof Techniques and Derivations

TODO: Content for this appendix.

Appendix B

CUDA and Triton Cookbook

TODO: Content for this appendix.

Appendix C

Linguistics Quick Reference

TODO: Content for this appendix.

Appendix D

Neuroscience Analogy Index

TODO: Content for this appendix.

Appendix E

Reproducibility and Debugging

TODO: Content for this appendix.

References

[1] "Prediction and Entropy of Printed English". In: (1950). URL: https://www.princeton.edu/~wbialek/rome/refs/shannon_51.pdf.

[2] "A Neural Probabilistic Language Model". In: (2003). Used a feedforward neural network over last n words to predict the next word in a sequence. URL: https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf.

[3] OpenAI. "Language Models are Unsupervised Multitask Learners". In: (2019). 1.5B parameters | Pioneered stage release. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[4] "Sennrich". In: (). arXiv: 1508.07909. URL: https://arxiv.org/abs/1508.07909.

[5] Google. "Moe". In: (). arXiv: 1701.06538. URL: https://arxiv.org/pdf/1701.06538.pdf.

[6] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, et al. "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding". In: (2020). Introduced expert parallelism, capacity factor, random routing for top-2 gating in MoE at scale. 600B parameter MoE model. arXiv: 2006.16668. URL: https://arxiv.org/abs/2006.16668.

[7] Google. "Switch Transformers". In: (). arXiv: 2101.03961. URL: https://arxiv.org/abs/2101.03961.

[8] Mistral. "Mixtral". In: (). arXiv: 2401.04088. URL: https://arxiv.org/pdf/2401.04088.pdf.

[9] "Deepseek V2". In: (). arXiv: 2405.04434. URL: https://arxiv.org/abs/2405.04434.

[10] "Deepseek V3". In: (). arXiv: 2412.19437. URL: https://arxiv.org/pdf/2412.19437.pdf.

[11] "Auxfree". In: (). arXiv: 2408.15664. URL: https://arxiv.org/abs/2408.15664.